

ĐẠI HỌC XÂY DỰNG HÀ NỘI  
KHOA CÔNG NGHỆ THÔNG TIN

o0o



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

NGÀNH: CÔNG NGHỆ THÔNG TIN

*Phát triển hệ thống blockchain riêng biệt cho một chuỗi  
doanh nghiệp (khai-chain)*

Cần Tất Dương

GIÁO VIÊN HƯỚNG DẪN:

Nguyễn Việt Nhật

Hà Nội, năm 2026

# Mục lục

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| Danh mục hình vẽ .....                                                | 3         |
| Danh mục bảng biểu .....                                              | 4         |
| Danh mục từ viết tắt, thuật ngữ.....                                  | 5         |
| Lời cam đoan .....                                                    | 7         |
| Lời cảm ơn .....                                                      | 8         |
| Lời nói đầu.....                                                      | 9         |
| <b>Chương 1: Giới thiệu đề tài .....</b>                              | <b>10</b> |
| 1.1 Lý do chọn đề tài . . . . .                                       | 10        |
| 1.1.1 Sự phức tạp và công kênh của các nền tảng blockchain hiện tại . | 10        |
| 1.1.2 Nhu cầu về một hệ thống blockchain thử nghiệm . . . . .         | 10        |
| 1.1.3 Nhu cầu thực tiễn của doanh nghiệp vừa và nhỏ . . . . .         | 11        |
| 1.2 Mục tiêu nghiên cứu . . . . .                                     | 11        |
| 1.2.1 Tối ưu hóa quá trình triển khai và khởi chạy node . . . . .     | 11        |
| 1.2.2 Xây dựng lõi blockchain chuyên biệt hóa cho chuỗi doanh nghiệp  | 11        |
| 1.3 Đối tượng và phạm vi nghiên cứu . . . . .                         | 12        |
| <b>Chương 2: Cơ sở lý thuyết và Công nghệ nền tảng.....</b>           | <b>13</b> |
| 2.1 Tổng quan về công nghệ Blockchain . . . . .                       | 13        |
| 2.1.1 Khái niệm và nguyên lý hoạt động cơ bản . . . . .               | 13        |
| 2.1.2 Các đặc trưng cốt lõi của Blockchain . . . . .                  | 13        |
| 2.1.3 Phân loại mạng lưới Blockchain . . . . .                        | 14        |
| 2.2 Cơ sở mật mã học trong Blockchain . . . . .                       | 14        |
| 2.2.1 Hàm băm mật mã . . . . .                                        | 14        |
| 2.2.2 Chữ ký số và Mã hóa bất đối xứng . . . . .                      | 14        |
| 2.3 Mạng ngang hàng (Peer-to-Peer Network) . . . . .                  | 16        |
| 2.3.1 Kiến trúc Client-Server so với P2P . . . . .                    | 16        |
| 2.3.2 Giao thức truyền bá thông điệp (Gossip Protocol) . . . . .      | 16        |
| 2.4 Cơ chế đồng thuận (Consensus Mechanisms) . . . . .                | 17        |
| 2.4.1 Bài toán Tướng Byzantine (Byzantine Generals Problem) . . . .   | 17        |
| 2.4.2 Các thuật toán đồng thuận truyền thống (PoW, PoS) . . . . .     | 17        |
| 2.4.3 Đồng thuận trong Private Blockchain (CFT và BFT) . . . . .      | 18        |
| 2.5 Hợp đồng thông minh (Smart Contract) . . . . .                    | 18        |
| 2.5.1 Khái niệm và đặc tính tất định (Determinism) . . . . .          | 18        |
| 2.5.2 Vòng đời thực thi của Hợp đồng thông minh . . . . .             | 19        |
| 2.6 Phân tích kiến trúc nền tảng Hyperledger Fabric . . . . .         | 20        |
| 2.6.1 Mô hình Execute-Order-Validate . . . . .                        | 20        |
| 2.6.2 Vấn đề thực tiễn và bài toán tối ưu tài nguyên . . . . .        | 20        |
| <b>Chương 3: Phân tích và Đặc tả yêu cầu hệ thống.....</b>            | <b>22</b> |
| 3.1 Kiến trúc tổng quát của Khoai Chain . . . . .                     | 22        |
| 3.1.1 Các thay đổi chính . . . . .                                    | 22        |
| 3.1.2 So sánh với Hyperledger Fabric . . . . .                        | 23        |
| 3.2 Mô hình các chức năng . . . . .                                   | 24        |

|                                                        |                                                                |           |
|--------------------------------------------------------|----------------------------------------------------------------|-----------|
| 3.3                                                    | Các trường hợp sử dụng . . . . .                               | 30        |
| 3.4                                                    | Yêu cầu phi chức năng . . . . .                                | 35        |
| <b>Chương 4: Thiết kế Kiến trúc Khoai Chain.....</b>   |                                                                | <b>36</b> |
| 4.1                                                    | Kiến trúc tổng thể hệ thống . . . . .                          | 36        |
| 4.2                                                    | Thiết kế Module P2P (Peer, Server, Handler, Message) . . . . . | 36        |
| 4.3                                                    | Thiết kế Core Blockchain . . . . .                             | 37        |
| 4.3.1                                                  | Cấu trúc dữ liệu Block và Transaction . . . . .                | 37        |
| 4.3.2                                                  | Luồng hoạt động của Mempool . . . . .                          | 37        |
| 4.4                                                    | Thiết kế hệ thống Smart Contract . . . . .                     | 37        |
| 4.5                                                    | Cơ chế đồng thuận . . . . .                                    | 38        |
| <b>Chương 5: Hướng dẫn sử dụng và Triển khai .....</b> |                                                                | <b>39</b> |
| 5.1                                                    | Khởi tạo môi trường và cấu hình . . . . .                      | 39        |
| 5.2                                                    | Chạy thử nghiệm bằng CLI và Docker . . . . .                   | 39        |
| 5.3                                                    | Kịch bản chạy thử nghiệm mẫu cho sinh viên . . . . .           | 39        |
| 5.4                                                    | Các chức năng đang phát triển và Hạn chế hiện tại . . . . .    | 40        |
| <b>Kết luận .....</b>                                  |                                                                | <b>41</b> |
|                                                        | Tóm tắt các chức năng đã chốt . . . . .                        | 41        |
|                                                        | Định hướng hoàn thiện mã nguồn . . . . .                       | 41        |
| <b>Tài liệu tham khảo .....</b>                        |                                                                | <b>43</b> |

## Danh sách hình vẽ

|     |                                                         |    |
|-----|---------------------------------------------------------|----|
| 1.1 | Công nghệ blockchain . . . . .                          | 10 |
| 2.1 | Client/Server vs P2P . . . . .                          | 16 |
| 2.2 | Vòng đời của smart-contract . . . . .                   | 19 |
| 3.1 | Sơ đồ Use Case mạng lưới Khoai Chain . . . . .          | 24 |
| 3.2 | Sơ đồ Sequence khởi tạo mạng lưới Khoai Chain . . . . . | 30 |
| 3.3 | Sơ đồ Sequence tạo Node mới Khoai Chain . . . . .       | 31 |
| 3.4 | Sơ đồ Sequence tạo Block Khoai Chain . . . . .          | 32 |
| 3.5 | Sơ đồ Sequence gửi giao dịch Khoai Chain . . . . .      | 33 |
| 3.6 | Sơ đồ Sequence truy vấn sổ cái Khoai Chain . . . . .    | 34 |

## Danh sách bảng

|     |                                                                      |    |
|-----|----------------------------------------------------------------------|----|
| 2.1 | So sánh thuật toán đồng thuận CFT và BFT . . . . .                   | 18 |
| 3.1 | So sánh đặc tính kỹ thuật giữa Hyperledger Fabric và Khoai Chain . . | 23 |
| 3.2 | Đặc tả chi tiết Use Case Khởi tạo mạng lưới Khoai Chain . . . . .    | 25 |
| 3.3 | Đặc tả chi tiết Use Case Tạo Node mới Khoai Chain . . . . .          | 26 |
| 3.4 | Đặc tả chi tiết Use Case Tạo Block (Đồng thuận khối) . . . . .       | 27 |
| 3.5 | Đặc tả chi tiết Use Case Gửi giao dịch . . . . .                     | 28 |
| 3.6 | Đặc tả chi tiết Use Case Truy vấn sổ cái . . . . .                   | 29 |

## Danh mục từ viết tắt, thuật ngữ

| STT | Từ viết tắt, thuật ngữ | Giải nghĩa                                                                                     |
|-----|------------------------|------------------------------------------------------------------------------------------------|
| 1   | API                    | Giao diện lập trình ứng dụng (Application Programming Interface)                               |
| 2   | BFT                    | Dung sai lỗi Byzantine (Byzantine Fault Tolerance)                                             |
| 3   | CA                     | Tổ chức cấp chứng chỉ (Certificate Authority)                                                  |
| 4   | CFT                    | Dung sai lỗi hỏng hóc (Crash Fault Tolerance)                                                  |
| 5   | CLI                    | Giao diện dòng lệnh (Command-Line Interface)                                                   |
| 6   | CPU                    | Bộ vi xử lý trung tâm                                                                          |
| 7   | DHT                    | Bảng băm phân tán (Distributed Hash Table)                                                     |
| 8   | DLT                    | Công nghệ sổ cái phân tán (Distributed Ledger Technology)                                      |
| 9   | DX                     | Trải nghiệm dành cho nhà phát triển (Developer Experience)                                     |
| 10  | ECDSA                  | Thuật toán chữ ký số dựa trên đường cong Elliptic (Elliptic Curve Digital Signature Algorithm) |
| 11  | EVM                    | Máy ảo Ethereum (Ethereum Virtual Machine)                                                     |
| 12  | I/O                    | Input/Output                                                                                   |
| 13  | IBFT                   | Istanbul Byzantine Fault Tolerance                                                             |
| 14  | ID                     | Mã định danh (Identifier)                                                                      |
| 15  | JSON                   | JavaScript Object Notation                                                                     |
| 16  | P2P                    | Mạng ngang hàng (Peer-to-Peer)                                                                 |
| 17  | PBFT                   | Practical Byzantine Fault Tolerance                                                            |
| 18  | PoA                    | Bằng chứng ủy quyền (Proof of Authority)                                                       |
| 19  | PoS                    | Bằng chứng cổ phần (Proof of Stake)                                                            |
| 20  | PoW                    | Bằng chứng công việc (Proof of Work)                                                           |
| 21  | RAM                    | Bộ nhớ trong                                                                                   |
| 22  | RPC                    | Remote Procedure Call                                                                          |
| 23  | RSA                    | Thuật toán mã hóa bất đối xứng Rivest-Shamir-Adleman                                           |

|    |         |                                                                |
|----|---------|----------------------------------------------------------------|
| 24 | SDK     | Bộ công cụ phát triển phần mềm (Software Development Kit)      |
| 25 | SHA-256 | Thuật toán băm an toàn 256-bit (Secure Hash Algorithm 256-bit) |
| 26 | SME     | Doanh nghiệp vừa và nhỏ (Small and Medium-sized Enterprises)   |
| 27 | SPOF    | Điểm lỗi duy nhất (Single Point of Failure)                    |
| 28 | TCP/IP  | Giao thức điều khiển truyền vận/Giao thức Internet             |
| 29 | TPS     | Tốc độ xử lý giao dịch (Transactions Per Second)               |
| 30 | VPS     | Máy chủ ảo (Virtual Private Server)                            |

## Lời cam đoan

Em xin cam đoan nội dung đồ án tốt nghiệp "Phát triển hệ thống blockchain riêng biệt cho một chuỗi doanh nghiệp (Khoai Chain)" là do em tự nghiên cứu, tìm hiểu và thực hiện dưới sự hướng dẫn của Thầy Nguyễn Việt Nhật.

Toàn bộ nội dung của đồ án không sao chép hay vay mượn trái phép từ các công trình khoa học khác. Các đoạn trích dẫn, số liệu và tài liệu tham khảo sử dụng trong báo cáo đều được trích dẫn nguồn gốc và ghi chú đầy đủ, rõ ràng trong danh mục tài liệu tham khảo.

Nếu phát hiện có bất kỳ sự gian lận hay sao chép nào, em xin chịu hoàn toàn trách nhiệm trước Hội đồng bảo vệ và nhà trường.

*Hà Nội, ngày ... tháng ... năm 2026*

**Sinh viên thực hiện**

Cần Tất Dương

## Lời cảm ơn

Để hoàn thành được đề án này, em xin gửi lời cảm ơn chân thành đến các thầy cô trong Khoa Công nghệ Thông tin - Trường Đại học Xây dựng Hà Nội đã tận tình giảng dạy và trang bị cho em những kiến thức nền tảng quý báu trong suốt bốn năm học vừa qua.

Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc đến Thầy Nguyễn Việt Nhật. Những bài giảng của thầy trong học phần An toàn bảo mật thông tin, cụ thể là việc đi sâu vào thuật toán mã hóa RSA, đã mở ra cho em những hiểu biết cốt lõi đầu tiên về hệ thống mật mã bất đối xứng (khóa công khai và khóa bí mật). Nền tảng tư duy vững chắc về bảo mật đó chính là kim chỉ nam giúp em có khả năng tự nghiên cứu, áp dụng chữ ký số đường cong Elliptic (Ed25519) [8] và trực tiếp lập trình cơ chế xác thực giao dịch an toàn cho lõi hệ thống Khoai Chain ngày hôm nay.

Bên cạnh đó, em cũng xin gửi lời cảm ơn chân thành đến gia đình, những người bạn và các anh chị đồng nghiệp đã luôn động viên, hỗ trợ và đóng góp các ý kiến quý báu trong suốt quá trình em phát triển mã nguồn cũng như hoàn thiện cuốn báo cáo này.

**Sinh viên thực hiện**

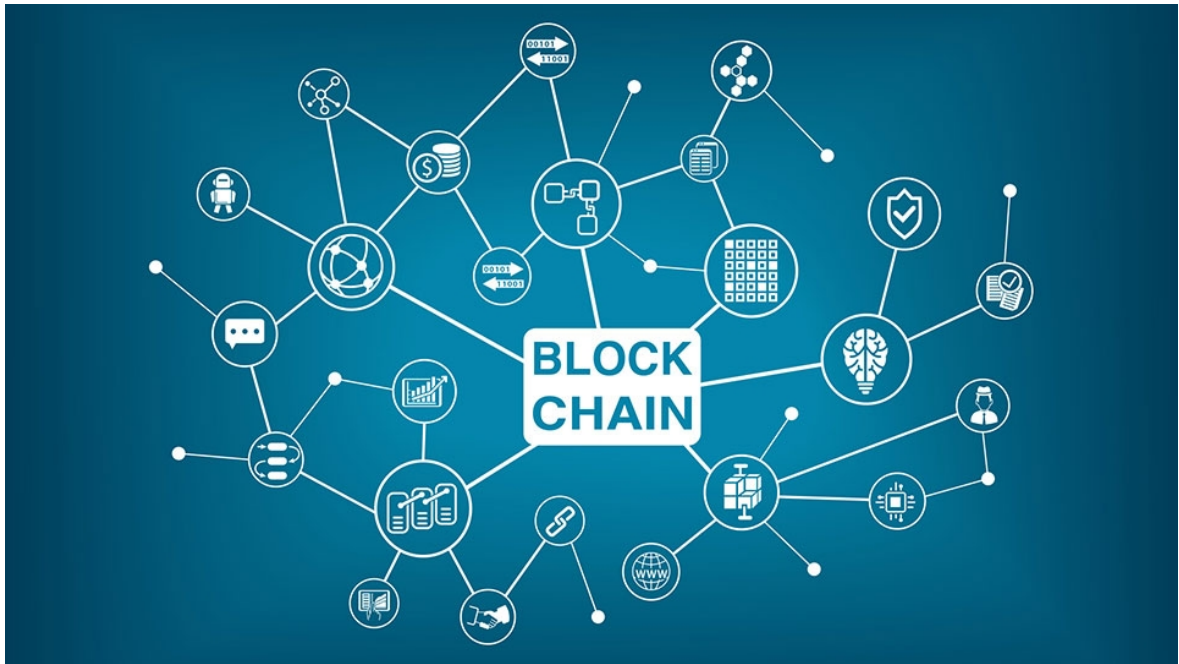
Cán Tất Dương

## Lời nói đầu

Công nghệ sổ cái phân tán (Blockchain) đang chứng minh được tiềm năng to lớn trong việc giải quyết các bài toán minh bạch hóa và bảo mật dữ liệu. Tuy nhiên, việc ứng dụng các nền tảng công nghiệp đồ sộ vào môi trường doanh nghiệp vừa và nhỏ (SME) vẫn còn nhiều rào cản về hạ tầng và tài nguyên. Đó chính là động lực để em thực hiện đồ án "Phát triển hệ thống blockchain riêng biệt cho một chuỗi doanh nghiệp (Khoai Chain)", với mục tiêu xây dựng một kiến trúc lõi tinh gọn nhưng vẫn đảm bảo tính toàn vẹn và bất biến của dữ liệu.

Mặc dù đã nỗ lực hết mình, nhưng do giới hạn về mặt thời gian và kinh nghiệm thực tiễn, đồ án chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự đánh giá và những lời góp ý quý báu từ Hội đồng bảo vệ để hệ thống được hoàn thiện hơn trong tương lai.

# Chương 1: Giới thiệu đề tài



Hình 1.1: Công nghệ blockchain

### 1.1. Lý do chọn đề tài

### 1.1.1. Sự phức tạp và công kênh của các nền tảng blockchain hiện tại

Từ thực tiễn triển khai các bài toán nghiệp vụ phức tạp, điển hình như hệ thống quản lý chuỗi cung ứng thuốc trên nền tảng như Hyperledger Fabric, công nghệ này bộc lộ rõ những rào cản lớn về mặt kỹ thuật và vận hành. Cụ thể, kiến trúc của Fabric đòi hỏi tài nguyên phần cứng đồ sộ để duy trì các thành phần độc lập (như Orderer, Peer, CA) và yêu cầu thiết lập hệ thống chứng chỉ số phức tạp [2]. Quá trình cấu hình, gỡ lỗi và triển khai mạng lưới tốn rất nhiều thời gian, trở thành một nút thắt lớn (bottleneck) cho các nhà phát triển. Sự cồng kềnh này hoàn toàn không phù hợp với các mô hình thử nghiệm nhanh, các dự án giáo dục, hoặc các doanh nghiệp có hạn chế ngân sách, từ đó đặt ra yêu cầu cấp thiết về một nền tảng blockchain tinh gọn hơn nhưng vẫn đảm bảo tính bất biến của dữ liệu.

### 1.1.2. Nhu cầu về một hệ thống blockchain thử nghiệm

Từ góc độ nghiên cứu và giáo dục, việc tiếp cận kiến trúc blockchain doanh nghiệp hiện nay gặp rất nhiều trở ngại do yêu cầu khắt khe về phần cứng. Người học và nhà phát triển thiếu đi một môi trường thử nghiệm tinh gọn, nơi họ có thể nhanh chóng khởi tạo một mạng lưới blockchain độc lập ngay trên máy tính cá nhân. Nhu cầu thực tế đòi hỏi một hệ thống được đóng gói nhẹ nhàng, đi kèm với các bộ công cụ phát triển (SDK) được tách biệt rõ ràng. Điều này giúp sinh viên và lập trình viên tập trung hoàn toàn vào việc xây dựng logic nghiệp vụ thông qua Hợp đồng thông minh (Smart

Contract), thay vì tốn thời gian xử lý các cấu hình mạng ngang hàng (P2P) hay cơ chế đồng thuận ở tầng lõi.

### **1.1.3. Nhu cầu thực tiễn của doanh nghiệp vừa và nhỏ**

Các doanh nghiệp vừa và nhỏ (SME) hoặc các chuỗi cung ứng quy mô nội bộ luôn có nhu cầu ứng dụng blockchain để giải quyết bài toán minh bạch hóa dữ liệu, chống giả mạo thông tin và truy xuất nguồn gốc. Mặc dù vậy, bài toán chi phí hạ tầng của các giải pháp chuẩn công nghiệp thường vượt quá khả năng đáp ứng của họ. Nhóm doanh nghiệp này cần một giải pháp blockchain được thiết kế tối giản nhưng vẫn giữ nguyên các đặc tính cốt lõi của sổ cái phân tán. Việc phát triển một hệ thống blockchain riêng biệt, có khả năng "cắm và chạy" (plug-and-play), tiêu tốn ít tài nguyên và dễ dàng tích hợp vào cơ sở hạ tầng máy chủ hiện có, sẽ giải quyết triệt để rào cản chi phí đồng thời bảo vệ an toàn cho các dữ liệu nhạy cảm của doanh nghiệp.

## **1.2. Mục tiêu nghiên cứu**

Đề tài hướng tới việc giải quyết những rào cản kỹ thuật của các nền tảng sổ cái phân tán truyền thống bằng cách xây dựng một hệ thống tinh gọn. Mục tiêu tổng thể được chia thành hai khía cạnh chính nhằm phục vụ cả môi trường nghiên cứu học thuật lẫn ứng dụng thực tiễn.

### **1.2.1. Tối ưu hóa quá trình triển khai và khởi chạy node**

Hệ thống được thiết kế để giảm thiểu tối đa các bước thiết lập môi trường phức tạp. Bằng cách phát triển các công cụ giao diện dòng lệnh (CLI framework) và đóng gói toàn bộ quy trình khởi chạy dưới dạng các container, mục tiêu là cho phép người dùng có thể kích hoạt một node mạng lưới độc lập chỉ bằng vài thao tác cơ bản. Quá trình triển khai tự động này loại bỏ sự phụ thuộc vào các dịch vụ hạ tầng trung gian, giúp việc kết nối vào mạng ngang hàng (P2P), thử nghiệm cục bộ và gỡ lỗi diễn ra nhanh chóng. Điều này đặc biệt hữu ích trong việc tiết kiệm tài nguyên tính toán, cho phép chạy mượt mà ngay trên các thiết bị máy tính cá nhân tiêu chuẩn.

### **1.2.2. Xây dựng lõi blockchain chuyên biệt hóa cho chuỗi doanh nghiệp**

Mục tiêu trọng tâm là phát triển một lõi (core blockchain) được tinh chỉnh trực tiếp cho các luồng nghiệp vụ nội bộ. Kiến trúc lõi sẽ tự chủ hoàn toàn các thành phần như quản lý mempool, đồng bộ block, giao thức P2P và định tuyến giao dịch (contract router). Bằng cách cung cấp một môi trường thực thi được cô lập và tách biệt qua bộ SDK, các nhà phát triển có thể trực tiếp ánh xạ các quy tắc kinh doanh thành Hợp đồng thông minh mà không làm ảnh hưởng đến tính ổn định của toàn mạng. Khác với kiến trúc phục vụ đám đông, hệ thống lõi này chuyên biệt hóa cho một nhóm các định chế hoặc đối tác kinh doanh đã được xác thực, đảm bảo giao dịch được lưu vết minh bạch, nhất quán nhưng vẫn nằm trong phạm vi giới hạn an toàn của chuỗi.

### 1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là công nghệ sổ cái phân tán (Distributed Ledger Technology), mạng ngang hàng (P2P), và cơ chế thực thi Hợp đồng thông minh (Smart Contract).

Phạm vi nghiên cứu tập trung vào việc thiết kế và xây dựng lõi hệ thống Khoai Chain từ đầu, giới hạn trong môi trường mạng nội bộ (private/consortium) phục vụ chuỗi doanh nghiệp vừa và nhỏ hoặc mục đích giáo dục. Đề tài không tập trung giải quyết các bài toán của public blockchain như phát hành tiền mã hóa (tokenomics), khai thác (mining), hay khả năng mở rộng hàng triệu người dùng.

## Chương 2: Cơ sở lý thuyết và Công nghệ nền tảng

### 2.1. Tổng quan về công nghệ Blockchain

#### 2.1.1. Khái niệm và nguyên lý hoạt động cơ bản

Blockchain là một công nghệ sổ cái phân tán (Distributed Ledger Technology - DLT) cho phép ghi lại các giao dịch một cách an toàn, minh bạch và không thể đảo ngược trên một mạng lưới máy tính ngang hàng (P2P). Thay vì lưu trữ dữ liệu tại một máy chủ trung tâm, dữ liệu của blockchain được nhân bản và đồng bộ hóa tại tất cả các nút tham gia mạng lưới [1].

Cấu trúc lưu trữ dữ liệu của Blockchain bao gồm chuỗi các khối (Blocks) liên kết tuyến tính với nhau theo thời gian. Mỗi khối chứa hai thành phần chính:

- **Block Header:** Chứa siêu dữ liệu (metadata) quản lý khối, bao gồm:
  - + **Version:** Phiên bản của giao thức blockchain.
  - + **Previous Hash:** Mã băm của toàn bộ header khối trước đó. Đây là sợi xích toán học liên kết các khối. Nếu dữ liệu khối trước bị sửa đổi, mã băm này thay đổi và làm gãy toàn bộ chuỗi phía sau.
  - + **Merkle Root Hash:** Mã băm gốc của cây Merkle, đại diện cho toàn bộ giao dịch trong khối [1].
  - + **Timestamp:** Thời gian khối được tạo ra.
  - + **Nonce / Bits:** Các tham số phục vụ cho cơ chế đồng thuận (ví dụ trong PoW).
- **Block Body (Thân khối):** Chứa danh sách chi tiết các giao dịch thực tế đã được xác thực và đóng gói vào khối.

#### 2.1.2. Các đặc trưng cốt lõi của Blockchain

- **Tính phân tán (Decentralization):** Hệ thống loại bỏ thực thể trung tâm quản lý. Quyền lực ghi và xác thực sổ cái được chia đều cho các nút trong mạng theo luật đồng thuận, giúp loại bỏ điểm lỗi duy nhất (SPOF) [1].
- **Tính minh bạch (Transparency):** Mọi tài khoản tham gia mạng lưới (đặc biệt là Public) đều có quyền truy cập, tra cứu và kiểm toán toàn bộ lịch sử giao dịch từ khối khởi thủy (Genesis Block) đến khối hiện tại.
- **Tính bất biến (Immutability):** Dữ liệu một khi đã được đồng thuận ghi vào chuỗi thì không thể sửa đổi hoặc xóa bỏ. Muốn thay đổi một khối, kẻ tấn công phải thay đổi tất cả các khối sau nó và chiếm trên 51% sức mạnh toán học hoặc quyền kiểm soát mạng lưới (điều này bất khả thi về mặt chi phí trong các mạng lớn).
- **Khả năng truy xuất (Traceability):** Nhờ cơ chế liên kết mã băm và mốc thời gian (timestamp) cố định, mọi thực thể dữ liệu hoặc tài sản số hóa lưu trữ trên blockchain đều có thể được truy vết nguồn gốc dịch chuyển một cách chính xác qua từng địa chỉ ví.

### 2.1.3. Phân loại mạng lưới Blockchain

- **Public Blockchain (Mạng công khai):** Bất kỳ ai cũng có thể tham gia đọc, ghi dữ liệu và tham gia vào quá trình đồng thuận (ví dụ: Bitcoin, Ethereum). Mạng này có tính phi tập trung tối đa nhưng tốc độ xử lý giao dịch (TPS) thấp và chi phí vận hành lớn.
- **Private Blockchain (Mạng tư nhân):** Quyền đọc, ghi và đồng thuận hoàn toàn thuộc về một tổ chức duy nhất quản lý. Tốc độ xử lý rất nhanh, bảo mật nội bộ cao, nhưng mang tính tập trung cao.
- **Consortium Blockchain (Mạng liên minh):** Quyền kiểm soát mạng lưới được chia sẻ cho một nhóm các tổ chức hoặc doanh nghiệp có lợi ích chung (ví dụ: liên minh ngân hàng, chuỗi cung ứng).

**Mối liên hệ với đề tài:** Đối với các bài toán chuỗi doanh nghiệp, Consortium/Private Blockchain (như Hyperledger Fabric) là mô hình tối ưu nhất. Nó vừa giữ được tính minh bạch, bất biến của blockchain giữa các bên đối tác, vừa giải quyết được bài toán bảo mật dữ liệu kinh doanh, kiểm soát quyền truy cập nghiêm ngặt và đạt hiệu năng xử lý cao (TPS lớn, độ trễ thấp) đáp ứng quy mô công nghiệp.

## 2.2. Cơ sở mật mã học trong Blockchain

### 2.2.1. Hàm băm mật mã

Hàm băm mật mã là một thuật toán toán học chuyển đổi một đầu vào dữ liệu có kích thước bất kỳ thành một chuỗi ký tự đầu ra có độ dài cố định. Trong các hệ thống blockchain, thuật toán SHA-256 (Secure Hash Algorithm 256-bit, dùng trong Bitcoin) hoặc Keccak256 (dùng trong Ethereum) được áp dụng phổ biến. Một hàm băm mật mã học tiêu chuẩn bắt buộc phải đảm bảo 3 đặc tính cốt lõi sau:

- **Tính tất định (Determinism):** Với cùng một thông điệp đầu vào  $M$ , hàm băm luôn luôn trả về một kết quả đầu ra  $H(M)$  duy nhất, không thay đổi qua các lần tính toán.
- **Tính một chiều:** Khi biết trước một giá trị băm  $Y$ , về mặt toán học cực kỳ khó khăn (bất khả thi trong thời gian thực) để tìm lại thông điệp gốc  $X$  sao cho  $H(X) = Y$ . Cách duy nhất là thử sai ngẫu nhiên (Brute-force).
- **Tính kháng va chạm:** Cực kỳ khó để tìm thấy hai thông điệp khác nhau hoàn toàn  $X_1$  và  $X_2$  mà khi băm lại cho ra cùng một kết quả ( $H(X_1) = H(X_2)$ ). Ngoài ra, hàm băm có hiệu ứng tuyết lở (Avalanche effect): chỉ cần thay đổi 1 bit ở đầu vào, toàn bộ chuỗi đầu ra sẽ thay đổi hoàn toàn [7].

### 2.2.2. Chữ ký số và Mã hóa bất đối xứng

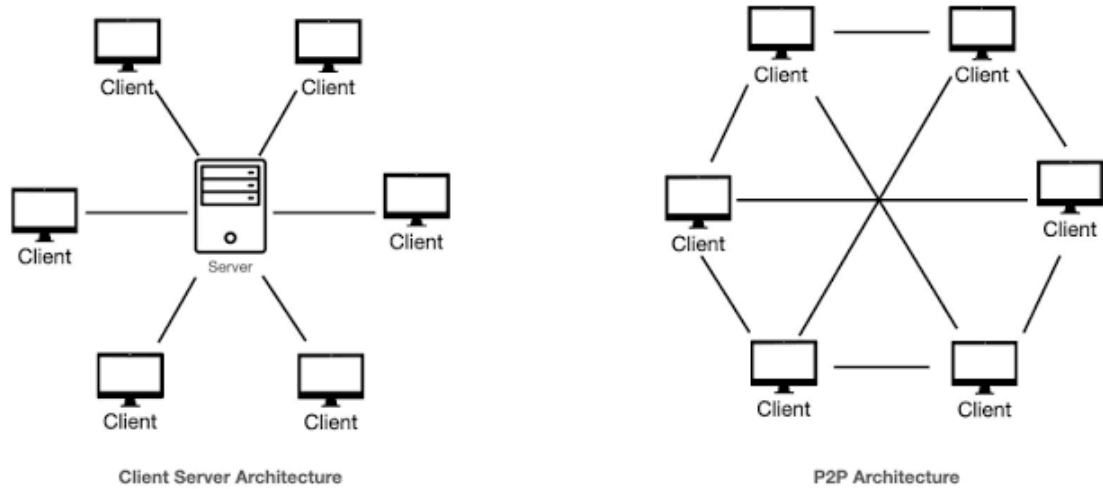
Mã hóa bất đối xứng sử dụng một cặp khóa toán học liên kết chặt chẽ với nhau: Private Key dùng để tạo chữ ký số và Khóa công khai (Public Key) dùng để xác thực chữ ký hoặc định danh địa chỉ ví.

Trong Blockchain, thuật toán ECDSA - Thuật toán chữ ký số dựa trên đường cong elliptic - được áp dụng để tối ưu hóa kích thước khóa và tốc độ tính toán so với RSA truyền thống.

- **Quy trình ký (Signing):** Khi người dùng muốn thực hiện giao dịch, họ dùng Private Key của mình kết hợp với mã băm của nội dung giao dịch thông qua thuật toán ECDSA để tạo ra một chuỗi chữ ký số (*Sig*).
- **Quy trình xác thực (Verification):** Các nút trong mạng nhận được giao dịch sẽ dùng Public Key của người gửi (được công khai) để giải mã và kiểm tra tính hợp lệ của *Sig*. Quy trình này chứng minh người gửi sở hữu tài khoản và đồng ý chuyển tiền mà không cần tiết lộ Private Key cho mạng lưới, ngăn chặn hoàn toàn việc giả mạo giao dịch.

## 2.3. Mạng ngang hàng (Peer-to-Peer Network)

### 2.3.1. Kiến trúc Client-Server so với P2P



Hình 2.1: Client/Server vs P2P

- **Kiến trúc Client-Server:** Dữ liệu tập trung hoàn toàn tại hệ thống máy chủ (Server). Các máy trạm (Client) muốn tương tác đều phải gửi yêu cầu về Server. Kiến trúc này chịu nhược điểm SPOF (Single Point of Failure): nếu máy chủ bị sập, bị tấn công DDoS, hoặc bị hack dữ liệu, toàn bộ hệ thống sẽ tê liệt và dữ liệu có nguy cơ bị thao túng hoàn toàn.
- **Kiến trúc P2P (Peer-to-Peer):** Mạng lưới bao gồm các nút (Peer) có vai trò và quyền hạn ngang hàng nhau. Mỗi nút vừa hoạt động như một Client (yêu cầu dữ liệu) vừa như một Server (cung cấp, lưu trữ, xác thực dữ liệu). Khi một số nút bị ngắt kết nối hoặc bị tấn công, hệ thống vẫn vận hành bình thường nhờ các bản sao tại các nút còn lại, loại bỏ hoàn toàn bài toán SPOF.

### 2.3.2. Giao thức truyền bá thông điệp (Gossip Protocol)

Gossip Protocol là cơ chế giao tiếp chính giúp các nút trong mạng lưới P2P phi tập trung có thể phát tán thông tin (như giao dịch mới hoặc khối mới tạo) ra toàn mạng một cách nhanh chóng và tiết kiệm băng thông.

- **Nguyên lý hoạt động:** Khi một nút nhận được một thông điệp mới, nó sẽ chọn ngẫu nhiên một số lượng nút  $K$  hàng xóm (Fanout) để chuyển tiếp thông điệp đó. Các nút nhận tiếp theo lại lặp lại quy trình chọn ngẫu nhiên này.
- **Hiệu quả:** Thông tin được lan truyền theo cấp số nhân. Chỉ trong một thời gian cực ngắn với độ phức tạp thời gian là  $O(\log N)$  [6] (với  $N$  là tổng số nút), toàn bộ mạng lưới rộng lớn sẽ đạt được trạng thái đồng bộ thông tin mà không cần một máy chủ điều phối trung tâm điều hướng luồng tin.

## 2.4. Cơ chế đồng thuận (Consensus Mechanisms)

### 2.4.1. Bài toán Tướng Byzantine (Byzantine Generals Problem)

Được đặt ra bởi Lamport, Shostak và Pease vào năm 1982 [3], bài toán mô tả một nhóm các vị tướng Byzantine vây hãm một thành phố địch. Họ liên lạc với nhau chỉ bằng tin nhắn messenger. Các tướng phải đồng thuận về một kế hoạch duy nhất: cùng Tấn công hoặc cùng Rút lui. Nếu có một vài tướng phản bội (hoặc tin nhắn bị thất lạc), họ sẽ cố tình gửi các thông điệp mâu thuẫn (gửi "Tấn công" cho tướng này nhưng gửi "Rút lui" cho tướng khác) nhằm làm chia rẽ và thất bại quân đội.

**Ý nghĩa trong Blockchain:** Đây là mô hình hóa thách thức đạt được sự đồng thuận dữ liệu trong một hệ thống phân tán, nơi các nút tham gia mạng có thể bị lỗi kỹ thuật (crash), mất kết nối, hoặc tệ hơn là chủ động gian lận, gửi dữ liệu sai lệch để trục lợi.

### 2.4.2. Các thuật toán đồng thuận truyền thống (PoW, PoS)

- **Proof of Work (PoW - Bằng chứng công việc):** Các nút phải cạnh tranh giải một bài toán toán học cực khó bằng sức mạnh phần cứng máy tính (tìm giá trị Nonce thích hợp). Người đầu tiên giải được sẽ giành quyền đóng gói khối và nhận phần thưởng [1]. PoW bảo mật rất cao nhưng tiêu tốn tài nguyên điện năng khổng lồ và tốc độ xử lý giao dịch rất chậm (Bitcoin chỉ khoảng 7 TPS).
- **Proof of Stake (PoS - Bằng chứng cổ phần):** Thay thế việc khai thác bằng phần cứng bằng việc đặt cọc (stake) tiền điện tử. Hệ thống dựa vào tỷ lệ cổ phần đặt cọc để chọn ngẫu nhiên nút có quyền tạo khối mới. PoS tiết kiệm năng lượng hơn PoW, tốc độ nhanh hơn, nhưng có nguy cơ dẫn đến việc tích lũy tài sản (người giàu càng giàu và nắm quyền kiểm soát mạng).
- **Proof of Authority (PoA - Bằng chứng ủy quyền):** Thay vì sử dụng năng lực tính toán hay lượng tiền đặt cọc, cơ chế này dựa trên danh tính và uy tín của các nút xác thực (Validators). Chỉ một nhóm các nút đã được kiểm duyệt, định danh và cấp phép từ trước mới có quyền đóng gói khối. PoA mang lại tốc độ xử lý giao dịch (TPS) cực kỳ cao, gần như không tiêu tốn tài nguyên phần cứng và rất phù hợp cho các mạng Private hoặc Consortium Blockchain, tuy nhiên phải đánh đổi bằng việc mang tính tập trung cao hơn so với PoW hay PoS.

### 2.4.3. Đồng thuận trong Private Blockchain (CFT và BFT)

Hệ thống chuỗi doanh nghiệp yêu cầu tốc độ xử lý cao và các nút tham gia đều được định danh rõ ràng. Do đó, họ áp dụng hai nhóm thuật toán tối ưu sau:

| Đặc tính so sánh             | Nhóm lỗi hồng học                                                                                                                                                           | Nhóm lỗi Byzantine                                                                                                                                      |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Bản chất chống lỗi</b>    | Chỉ chịu được lỗi nút bị sập nguồn, mất mạng (đột ngột dừng hoạt động).                                                                                                     | Chịu được cả lỗi nút bị sập VÀ nút cố tình gian lận, gửi dữ liệu sai.                                                                                   |
| <b>Thuật toán đại diện</b>   | Raft [5], Paxos.                                                                                                                                                            | PBFT [4], IBFT.                                                                                                                                         |
| <b>Cơ chế hoạt động</b>      | Bầu ra một nút Leader (Trưởng nhóm). Mọi giao dịch gửi tới Leader, Leader ghi logs và đồng bộ xuống các nút Follower. Nếu Leader sập, các nút còn lại bầu nhanh Leader mới. | Trải qua 3 giai đoạn đồng thuận nghiêm ngặt công khai: Pre-prepare, Prepare, và Commit thông qua việc trao đổi thông điệp toàn vẹn giữa tất cả các nút. |
| <b>Khả năng chịu lỗi</b>     | Chịu lỗi tối đa lên tới $F = \frac{N-1}{2}$ (Mạng cần $2F + 1$ nút để hoạt động).                                                                                           | Chịu lỗi tối đa thấp hơn: $F = \frac{N-1}{3}$ (Mạng cần tối thiểu $3F + 1$ nút để hoạt động).                                                           |
| <b>Ứng dụng doanh nghiệp</b> | Thích hợp cho môi trường nội bộ doanh nghiệp tin cậy cao, cần tốc độ cực nhanh.                                                                                             | Thích hợp cho môi trường Consortium (liên minh nhiều bên đối tác có sự nghi kị lẫn nhau).                                                               |

Bảng 2.1: So sánh thuật toán đồng thuận CFT và BFT

## 2.5. Hợp đồng thông minh (Smart Contract)

### 2.5.1. Khái niệm và đặc tính tất định (Determinism)

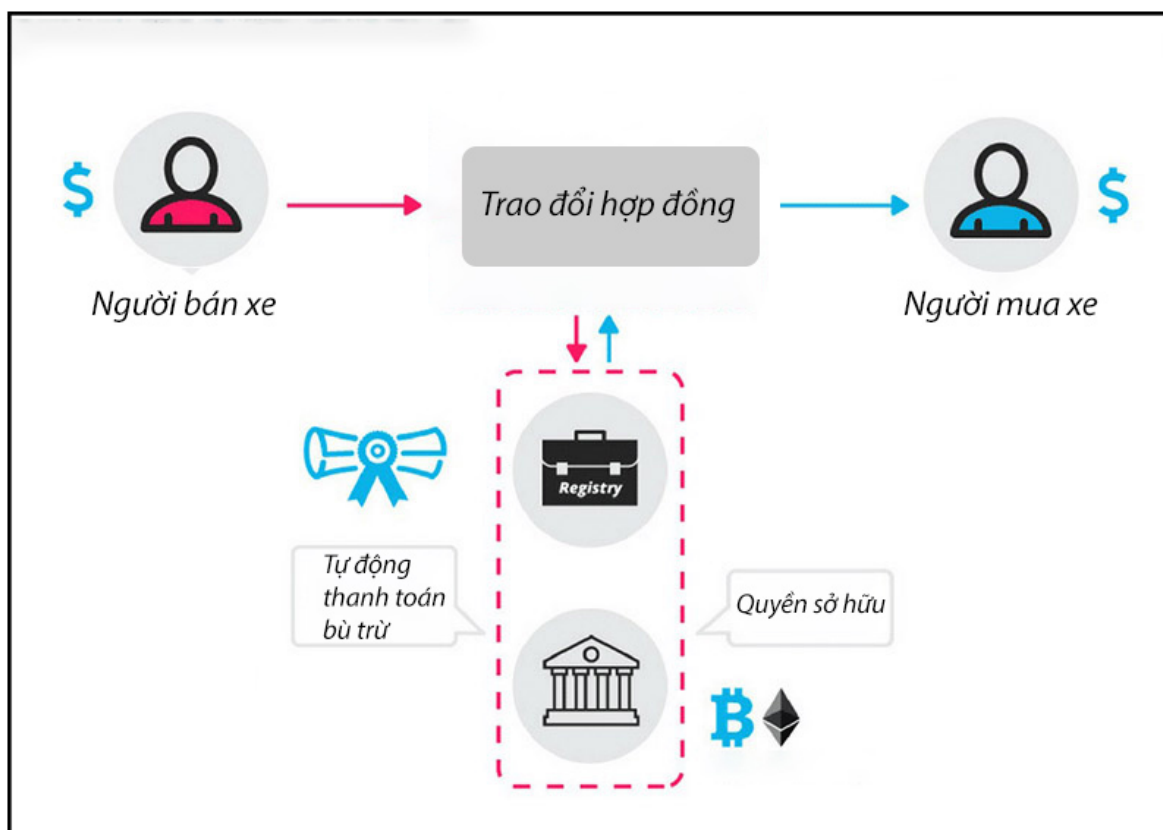
Hợp đồng thông minh là một chương trình máy tính (mã nguồn mã hóa) được triển khai trực tiếp và lưu trữ vĩnh viễn trên nền tảng Blockchain. Nó tự động thực thi các điều khoản thỏa thuận giữa các bên khi các điều kiện logic định trước được thỏa mãn mà không cần thông qua bên trung gian thứ ba.

**Đặc tính tất định:** Đây là đặc tính bắt buộc tối thượng của Smart Contract. Với cùng một trạng thái số cái đầu vào (State input) và cùng một tham số truyền vào hàm, việc thực thi Smart Contract tại bất kỳ nút nào trong mạng lưới, tại bất kỳ thời điểm nào, cũng đều phải trả về một kết quả đầu ra giống nhau 100% và cập nhật trạng thái số cái đồng nhất. Do đó, các hàm lấy thời gian thực của máy cục bộ, hay hàm tạo số ngẫu nhiên thuần túy đều bị cấm sử dụng trực tiếp trong mã nguồn của Smart Contract.

### 2.5.2. Vòng đời thực thi của Hợp đồng thông minh

Vòng đời thực thi của một Hợp đồng thông minh trải qua 5 bước nghiêm ngặt:

1. **Viết mã:** Lập trình viên viết mã nguồn hợp đồng bằng ngôn ngữ chuyên dụng (như Solidity trên Ethereum, hoặc Go/Java/Node.js cho Chaincode trên Hyperledger Fabric).
2. **Biên dịch:** Mã nguồn được biên dịch thành mã máy (Bytecode) để máy ảo (như EVM) hoặc môi trường thực thi (Docker container) có thể đọc hiểu.
3. **Triển khai:** Bytecode được đóng gói vào một giao dịch đặc biệt và gửi lên mạng lưới Blockchain. Sau khi được các nút đồng thuận và đóng khối thành công, hợp đồng sẽ có một Địa chỉ duy nhất (Contract Address) cố định trên chuỗi.
4. **Định tuyến gọi hàm:** Khi người dùng hoặc ứng dụng client muốn tương tác, họ gửi một giao dịch đến địa chỉ của hợp đồng kèm theo tên hàm và tham số đầu vào. Hệ thống mạng lưới sẽ định tuyến (route) yêu cầu này đến đúng môi trường cô lập chứa mã hợp đồng để kích hoạt thực thi.
5. **Cập nhật trạng thái:** Sau khi tính toán xong, kết quả thực thi tạo ra một trạng thái mới (World State thay đổi). Trạng thái mới này được phân phối đồng bộ, xác nhận qua cơ chế đồng thuận mạng và ghi vĩnh viễn vào sổ cái phân tán.



Hình 2.2: Vòng đời của smart-contract

## 2.6. Phân tích kiến trúc nền tảng Hyperledger Fabric

### 2.6.1. Mô hình Execute-Order-Validate

Các mạng Blockchain truyền thống như Bitcoin hay Ethereum sử dụng mô hình Thứ tự trước - Thực thi sau. Nghĩa là các giao dịch được gom lại, sắp xếp thứ tự và đóng khối trước bởi các thợ đào (Order), sau đó tất cả các nút trong mạng đồng loạt tải khối về rồi mới tự chạy thực thi lại mã nguồn để cập nhật sổ cái (Execute). Mô hình này bộc lộ điểm nghẽn hiệu năng nghiêm ngặt (tất cả các nút đều phải chạy lại mọi thứ) và không đảm bảo tính bảo mật thương mại. Hyperledger Fabric kiến tạo một kiến trúc đột phá hoàn toàn: Thực thi trước - Sắp xếp sau - Kiểm tra lại [2]. Quy trình gồm 3 bước riêng biệt phân rõ nhiệm vụ cho từng loại nút chuyên biệt:

- **Execute (Thực thi/Ký duyệt):** Ứng dụng gửi đề xuất giao dịch đến các nút ký duyệt chỉ định trước (Endorsing Nodes). Các nút này chạy thử nghiệm mã Smart Contract (gọi là Chaincode) một cách độc lập trong môi trường bảo mật (Docker) để tạo ra kết quả đọc/ghi (Read-Write Set) và ký số xác nhận mà không ghi ngay vào sổ cái.
- **Order (Sắp xếp thứ tự):** Ứng dụng thu thập đủ chữ ký phê duyệt từ bước 1 và gửi Read-Write Set đến dịch vụ sắp xếp (Ordering Service / Orderer Nodes). Các nút Orderer chịu trách nhiệm duy nhất là sắp xếp các giao dịch theo đúng trình tự thời gian và đóng gói chúng thành các khối (Blocks) mà không quan tâm bên trong giao dịch chạy logic gì. Điều này tối ưu hóa băng thông mạng cực lớn.
- **Validate (Xác thực lại):** Khối từ Orderer được phát tán tới các nút lưu trữ (Committing Nodes). Các nút này kiểm tra lại xem các chữ ký phê duyệt có hợp lệ và tuân thủ chính sách hay không, đồng thời kiểm tra xem có xung đột dữ liệu Read-Write (xung đột song song) hay không. Nếu hợp lệ, giao dịch chính thức được ghi (Commit) và cập nhật vào cơ sở dữ liệu trạng thái thế giới (World State).

### 2.6.2. Vấn đề thực tiễn và bài toán tối ưu tài nguyên

Mặc dù kiến trúc Execute-Order-Validate mang lại hiệu năng cao và phân quyền mạnh mẽ cho doanh nghiệp lớn, tuy nhiên, việc vận hành và triển khai một mạng Hyperledger Fabric chuẩn hóa trong thực tế đối diện với sự phức tạp và công kênh rất lớn về mặt kiến trúc tài nguyên phần cứng hệ thống [2]:

- **Hệ thống CA (Certificate Authority) đồ sộ:** Fabric quản lý định danh người dùng và các nút bằng chứng thư số X.509. Mỗi tổ chức (Organization) tham gia yêu cầu phải duy trì một hoặc nhiều cụm máy chủ CA độc lập, gây quá tải quản trị.
- **Cụm Orderer phức tạp:** Để đảm bảo tính sẵn sàng cao, dịch vụ Orderer phải chạy thuật toán Raft gồm nhiều nút cấu hình phức tạp, đòi hỏi cấu hình mạng ổn định và độ trễ cực thấp.
- **Cơ sở dữ liệu CouchDB/LevelDB phân tách:** Mỗi nút Peer phải gánh một cơ sở dữ liệu riêng để lưu trữ World State (thường là CouchDB để hỗ trợ truy vấn JSON nâng cao). Việc đồng bộ hóa dữ liệu liên tục giữa các container CouchDB tiêu tốn lượng RAM và I/O đĩa cứng cực kỳ lớn.

- **Rào cản doanh nghiệp vừa và nhỏ (SME):** Sự phức tạp của việc thiết lập hàng chục container Docker chạy đồng thời (gồm các Peer, Orderer, CA, CouchDB) đòi hỏi hạ tầng máy chủ mạnh mẽ và đội ngũ DevOps trình độ chuyên môn cao. Đối với các doanh nghiệp vừa và nhỏ (SME) hoặc các dự án có nguồn lực tài nguyên hạn chế, việc cấu hình và duy trì một hệ thống Fabric đầy đủ cấu hình tiêu chuẩn là một "gánh nặng chi phí" và lãng phí năng lực tính toán.

## Chương 3: Phân tích và Đặc tả yêu cầu hệ thống

### 3.1. Kiến trúc tổng quát của Khoai Chain

#### 3.1.1. Các thay đổi chính

Để giải quyết bài toán congestion của các nền tảng blockchain công nghiệp, Khoai Chain được thiết kế với triết lý tinh gọn (minimalist), tập trung vào cốt lõi của sổ cái phân tán và loại bỏ các thành phần không cần thiết cho quy mô doanh nghiệp vừa và nhỏ. Các thay đổi mang tính bước ngoặt bao gồm:

- **Tích hợp vai trò Node (Node Consolidation):** Thay vì phân rã hệ thống thành các nút độc lập (như Orderer Node để sắp xếp, Peer Node để lưu trữ, và CA để cấp phép), Khoai Chain tích hợp tất cả các chức năng đồng thuận, định tuyến giao dịch và lưu trữ vào một module Node duy nhất. Điều này giúp giảm thiểu độ trễ giao tiếp qua mạng lưới và tiết kiệm tối đa RAM/CPU.
- **Cơ chế lưu trữ nhúng (Embedded Storage):** Thay vì yêu cầu thiết lập các container cơ sở dữ liệu bên ngoài (như CouchDB), Khoai Chain sử dụng cơ sở dữ liệu dạng Key-Value được nhúng trực tiếp vào lõi hệ thống. Quá trình đồng bộ trạng thái sổ cái diễn ra nội bộ, loại bỏ hoàn toàn các cấu hình kết nối mạng phức tạp giữa Core và Database.
- **Tách biệt hoàn toàn SDK Hợp đồng thông minh:** Mã nguồn thực thi (Smart Contract) được tách ra khỏi lõi mạng P2P thông qua bộ công cụ `khoaichain-sdk`. Sự thay đổi này giúp các nhà phát triển và sinh viên chỉ cần kế thừa các interface có sẵn để viết logic nghiệp vụ, biên dịch và nạp vào mạng lưới mà không cần hiểu sâu về kiến trúc lõi của Khoai Chain.
- **Lược bỏ cơ chế kinh tế:** Do hệ thống phục vụ chuỗi doanh nghiệp nội bộ đã có sự tin tưởng và định danh nhất định, toàn bộ cơ chế khai thác, phí giao dịch và phần thưởng khối đều được loại bỏ để tăng tốc độ đóng gói giao dịch.

### 3.1.2. So sánh với Hyperledger Fabric

Sự khác biệt cốt lõi giữa Khoai Chain và tiêu chuẩn công nghiệp Hyperledger Fabric được thể hiện chi tiết qua bảng đối chiếu sau:

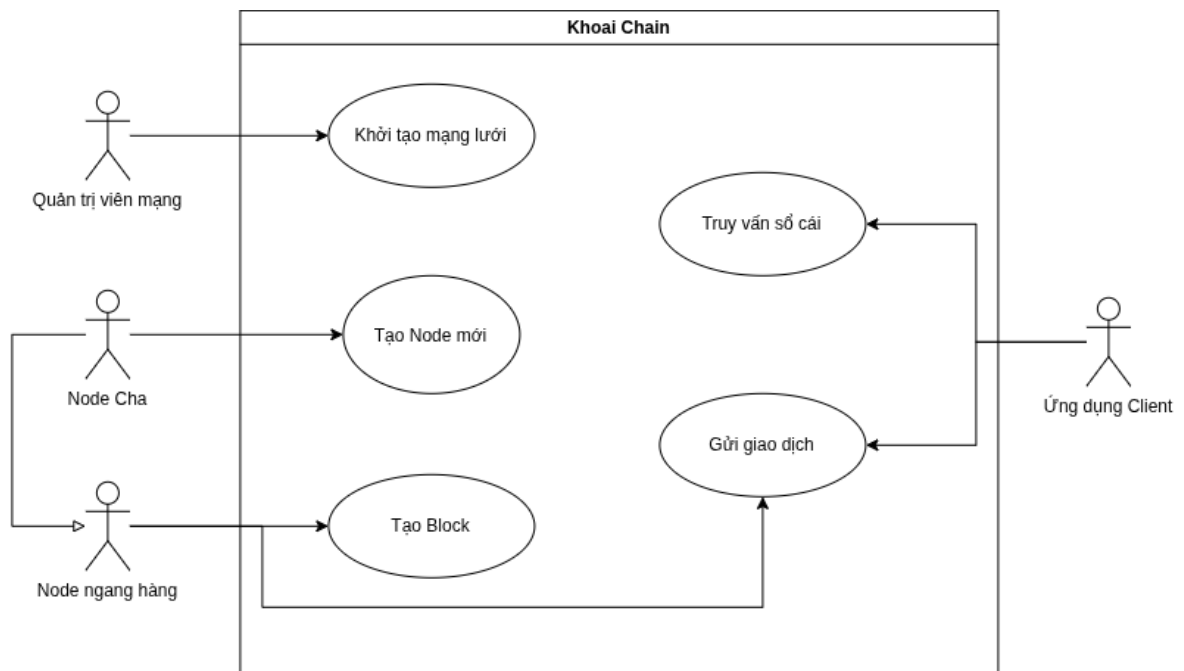
| Tiêu chí                   | Hyperledger Fabric                                                                        | Khoai Chain                                                                                  |
|----------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>Kiến trúc Node</b>      | Phân rã phức tạp (Peer, Orderer, CA hoạt động trên các vùng nhớ riêng).                   | Tích hợp (Tất cả vai trò xử lý, đồng thuận, lưu trữ nằm trong một tiến trình duy nhất).      |
| <b>Lưu trữ dữ liệu</b>     | Cần hệ quản trị CSDL độc lập (CouchDB/LevelDB chạy khác container).                       | Tích hợp CSDL Key-Value nhúng trực tiếp vào Core (Tiết kiệm I/O).                            |
| <b>Phát triển Contract</b> | Chaincode chạy trong môi trường Docker bị cô lập, cấu hình giao tiếp mạng phức tạp.       | Phát triển qua <code>khoaichain-sdk</code> độc lập, dễ dàng biên dịch và thực thi trực tiếp. |
| <b>Quản lý định danh</b>   | Sử dụng Certificate Authority (CA) với chứng chỉ X.509, yêu cầu duy trì máy chủ CA riêng. | Quản lý tinh gọn qua tệp cấu hình tĩnh (yaml) và cơ chế khóa xác thực nội bộ.                |
| <b>Mức độ triển khai</b>   | Rất nặng. Yêu cầu kiến thức DevOps sâu rộng, tốn nhiều cấu hình YAML/Docker Compose.      | Rất nhẹ. Khởi chạy thông qua CLI framework đơn giản hoặc một container duy nhất.             |
| <b>Đối tượng phù hợp</b>   | Tập đoàn lớn, liên minh đa quốc gia, ngân hàng.                                           | Sinh viên nghiên cứu, doanh nghiệp vừa và nhỏ (SME), hệ thống thử nghiệm cục bộ.             |

Bảng 3.1: So sánh đặc tính kỹ thuật giữa Hyperledger Fabric và Khoai Chain

### 3.2. Mô hình các chức năng

Sơ đồ Use Case tổng quát mô tả các tương tác cốt lõi giữa người dùng và hệ thống lõi của mạng lưới. Trong hệ thống này, các tác nhân được chia làm hai nhóm chính: nhóm quản trị hạ tầng (bao gồm Quản trị viên mạng, Node cha, Node ngang hàng) và nhóm người dùng cuối (Ứng dụng Client).

Cấu trúc phân quyền được thiết kế chặt chẽ nhằm đảm bảo tính phi tập trung nhưng vẫn kiểm soát được vòng đời sinh trưởng của các nút trong mạng. Chi tiết về luồng thực thi và các ràng buộc của từng chức năng được đặc tả cụ thể trong phần dưới đây.



Hình 3.1: Sơ đồ Use Case mạng lưới Khoai Chain

a) Khởi tạo mạng lưới

| Thuộc tính                 | Mô tả chi tiết                                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tên Use Case</b>        | Khởi tạo mạng lưới (Khởi tạo khối Genesis)                                                                                                                                                                                                                                                                                                                                                   |
| <b>Tác nhân</b>            | Quản trị viên mạng                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Tiền điều kiện</b>      | Hệ thống máy chủ được cài đặt mã nguồn lõi Khoai Chain. Chưa có dữ liệu sổ cái nào tồn tại.                                                                                                                                                                                                                                                                                                  |
| <b>Luồng sự kiện chính</b> | <ol style="list-style-type: none"> <li>1. Quản trị viên cấu hình các tham số mạng cơ bản (thời gian block, thuật toán đồng thuận) vào file cấu hình.</li> <li>2. Quản trị viên chạy lệnh khởi tạo trên CLI.</li> <li>3. Hệ thống tạo ra Khối Genesis (Khối số 0) chứa định danh của Quản trị viên.</li> <li>4. Hệ thống ghi Khối Genesis vào cơ sở dữ liệu cục bộ và mở cổng P2P.</li> </ol> |
| <b>Luồng ngoại lệ</b>      | <ul style="list-style-type: none"> <li>- Nếu định dạng file cấu hình sai: Hệ thống báo lỗi và dừng tiến trình.</li> <li>- Nếu cổng P2P đã bị chiếm dụng: Hệ thống báo lỗi không thể mở kết nối mạng.</li> </ul>                                                                                                                                                                              |
| <b>Hậu điều kiện</b>       | Mạng lưới được khởi động với 1 Nút gốc duy nhất. Sổ cái ở trạng thái sẵn sàng tiếp nhận Node mới.                                                                                                                                                                                                                                                                                            |

Bảng 3.2: Đặc tả chi tiết Use Case Khởi tạo mạng lưới Khoai Chain

b) Tạo Node mới

| Thuộc tính                 | Mô tả chi tiết                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tên Use Case</b>        | Tạo Node mới                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Tác nhân</b>            | Quản trị viên mạng, Node cha (Node quản lý)                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Tiền điều kiện</b>      | Mạng lưới đang hoạt động. Tác nhân phải có quyền hợp lệ trên chuỗi để thực hiện cấp phép.                                                                                                                                                                                                                                                                                                                                                 |
| <b>Luồng sự kiện chính</b> | <ol style="list-style-type: none"> <li>1. Tác nhân gửi yêu cầu tạo Node con thông qua SDK, kèm theo địa chỉ Public Key của Node mới.</li> <li>2. Hệ thống kiểm tra chữ ký và cấp bậc (Tier) của Tác nhân trong Hợp đồng thông minh.</li> <li>3. Hệ thống (bao gồm các Nút xác thực) thực hiện đồng thuận để ghi giao dịch cấp phép vào sổ cái.</li> <li>4. Nút mới khởi động, kết nối tới Node cha và bắt đầu đồng bộ dữ liệu.</li> </ol> |
| <b>Luồng ngoại lệ</b>      | <ul style="list-style-type: none"> <li>- Tác nhân không đủ quyền (không phải Node cha): Giao dịch bị từ chối, trả về lỗi "Unauthorized".</li> <li>- Mạng lưới không đạt đồng thuận: Giao dịch cấp phép bị hủy bỏ.</li> </ul>                                                                                                                                                                                                              |
| <b>Hậu điều kiện</b>       | Node mới được định danh hợp lệ trên sổ cái và có thể tham gia vào mạng lưới theo đúng cấp bậc được giao.                                                                                                                                                                                                                                                                                                                                  |

Bảng 3.3: Đặc tả chi tiết Use Case Tạo Node mới Khoai Chain

c) Tạo Block (Đồng thuận khối)

| Thuộc tính                 | Mô tả chi tiết                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tên Use Case</b>        | Tạo Block (Đồng thuận khối)                                                                                                                                                                                                                                                                                                                                                  |
| <b>Tác nhân</b>            | Nút xác thực (Validator Node)                                                                                                                                                                                                                                                                                                                                                |
| <b>Tiền điều kiện</b>      | Có ít nhất 1 giao dịch hợp lệ đang chờ trong Mempool. Nút đang ở trạng thái hoạt động.                                                                                                                                                                                                                                                                                       |
| <b>Luồng sự kiện chính</b> | <ol style="list-style-type: none"> <li>1. Nút đến lượt làm Trưởng nhóm lấy các giao dịch từ Mempool.</li> <li>2. Nút Trưởng nhóm đóng gói giao dịch thành Khối đề xuất và phát tán cho các Nút xác thực khác.</li> <li>3. Các Nút xác thực kiểm tra tính hợp lệ của Khối và phát tín hiệu bỏ phiếu.</li> <li>4. Khi đủ tỷ lệ phiếu đồng ý (100%), Khối được chốt.</li> </ol> |
| <b>Luồng ngoại lệ</b>      | <ul style="list-style-type: none"> <li>- Khối chứa giao dịch sai logic: Các nút từ chối bỏ phiếu, Khối bị hủy sau thời gian timeout.</li> <li>- Nút Trưởng nhóm mất kết nối: Mạng lưới tự động bầu Trưởng nhóm mới.</li> </ul>                                                                                                                                               |
| <b>Hậu điều kiện</b>       | Khối mới được nối vào chuỗi. Trạng thái cơ sở dữ liệu State DB được cập nhật.                                                                                                                                                                                                                                                                                                |

Bảng 3.4: Đặc tả chi tiết Use Case Tạo Block (Đồng thuận khối)

d) Gửi giao dịch

| Thuộc tính          | Mô tả chi tiết                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tên Use Case        | Gửi giao dịch                                                                                                                                                                                                                                                                                                                                                                  |
| Tác nhân            | Ứng dụng Client                                                                                                                                                                                                                                                                                                                                                                |
| Tiền điều kiện      | Client có Private Key hợp lệ. Kết nối mạng tới ít nhất 1 Nút đang mở.                                                                                                                                                                                                                                                                                                          |
| Luồng sự kiện chính | <ol style="list-style-type: none"> <li>1. Client tạo dữ liệu giao dịch nghiệp vụ (ví dụ: tạo lô thuốc) và ký số bằng Private Key.</li> <li>2. Client gửi gói tin giao dịch qua API/RPC tới Nút tiếp nhận.</li> <li>3. Nút tiếp nhận xác thực sơ bộ (chữ ký, định dạng) và đẩy vào Mempool.</li> <li>4. Nút tiếp nhận trả về mã băm (TxHash) cho Client để theo dõi.</li> </ol> |
| Luồng ngoại lệ      | <ul style="list-style-type: none"> <li>- Chữ ký không hợp lệ: Nút tiếp nhận trả về lỗi 400 Bad Request ngay lập tức.</li> <li>- Mempool đang đầy: Nút trả về lỗi 429 Too Many Requests.</li> </ul>                                                                                                                                                                             |
| Hậu điều kiện       | Giao dịch nằm trong Mempool, chờ được đóng gói vào Khối.                                                                                                                                                                                                                                                                                                                       |

Bảng 3.5: Đặc tả chi tiết Use Case Gửi giao dịch

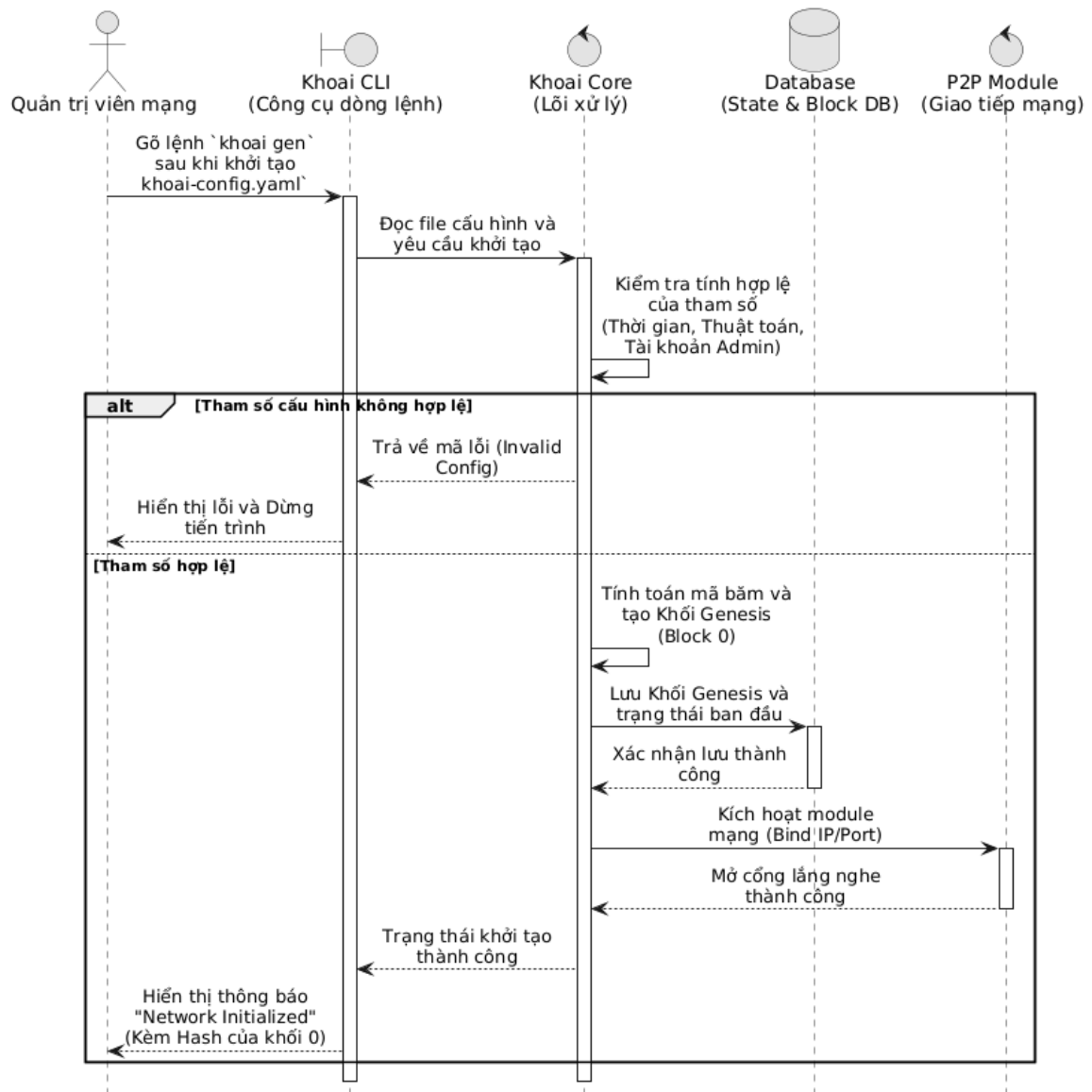
e) Truy vấn sổ cái

| Thuộc tính          | Mô tả chi tiết                                                                                                                                                                                                                                              |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tên Use Case        | Truy vấn sổ cái                                                                                                                                                                                                                                             |
| Tác nhân            | Ứng dụng Client                                                                                                                                                                                                                                             |
| Tiền điều kiện      | Client kết nối thành công tới Nút trong mạng lưới.                                                                                                                                                                                                          |
| Luồng sự kiện chính | <ol style="list-style-type: none"> <li>Client gửi yêu cầu đọc dữ liệu (ví dụ: truy xuất thông tin lô thuốc) tới Nút.</li> <li>Nút truy xuất dữ liệu từ State DB cục bộ của mình.</li> <li>Nút định dạng dữ liệu thành JSON và trả về cho Client.</li> </ol> |
| Luồng ngoại lệ      | <ul style="list-style-type: none"> <li>Dữ liệu không tồn tại: Nút trả về phản hồi rỗng hoặc thông báo "Not Found".</li> <li>Yêu cầu truy vấn quá lớn (quét toàn bộ chuỗi): Nút ngắt kết nối để tránh quá tải.</li> </ul>                                    |
| Hậu điều kiện       | Client nhận được dữ liệu trạng thái mới nhất từ sổ cái.                                                                                                                                                                                                     |

Bảng 3.6: Đặc tả chi tiết Use Case Truy vấn sổ cái

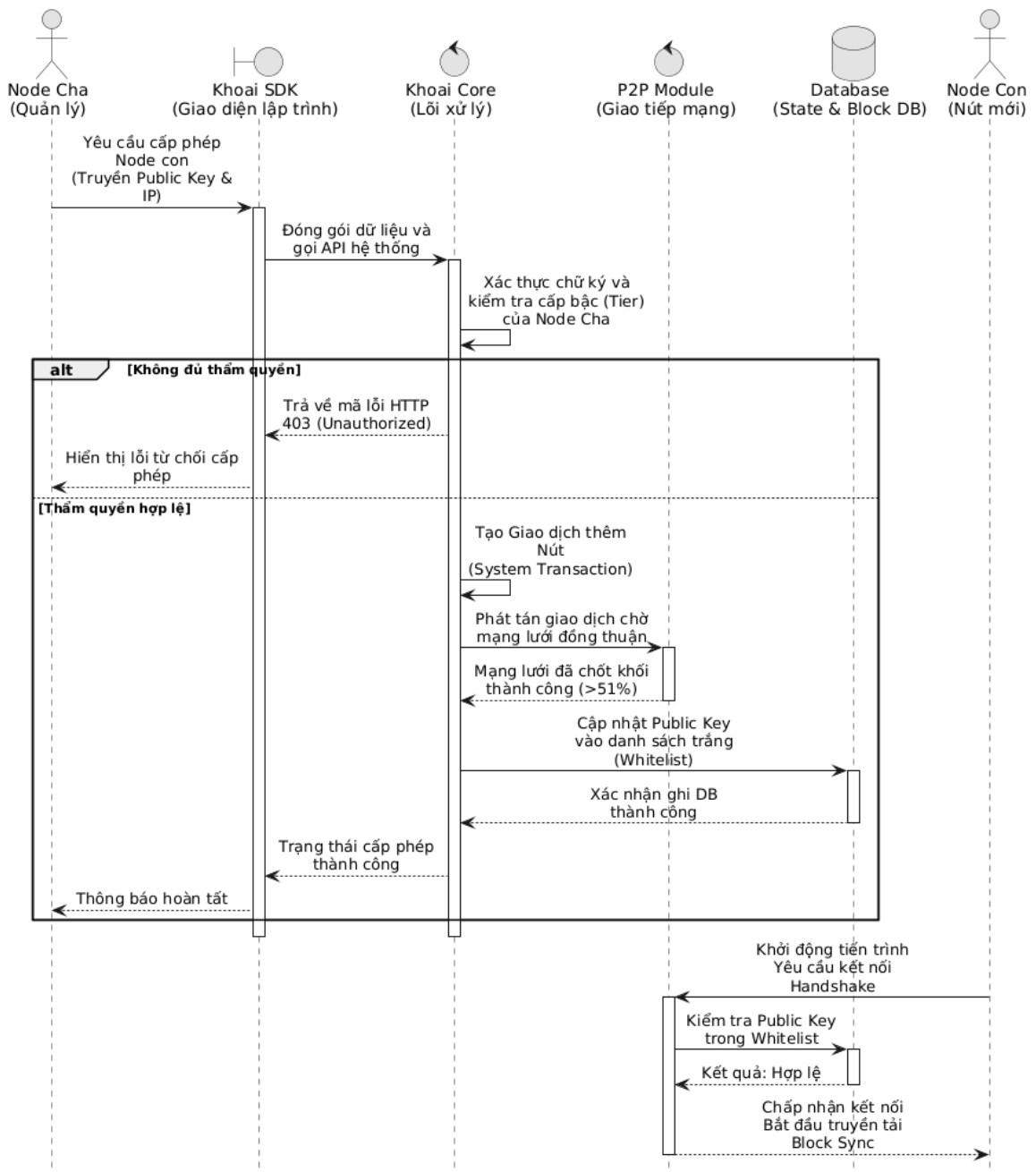
### 3.3. Các trường hợp sử dụng

#### a) Khởi tạo mạng lưới



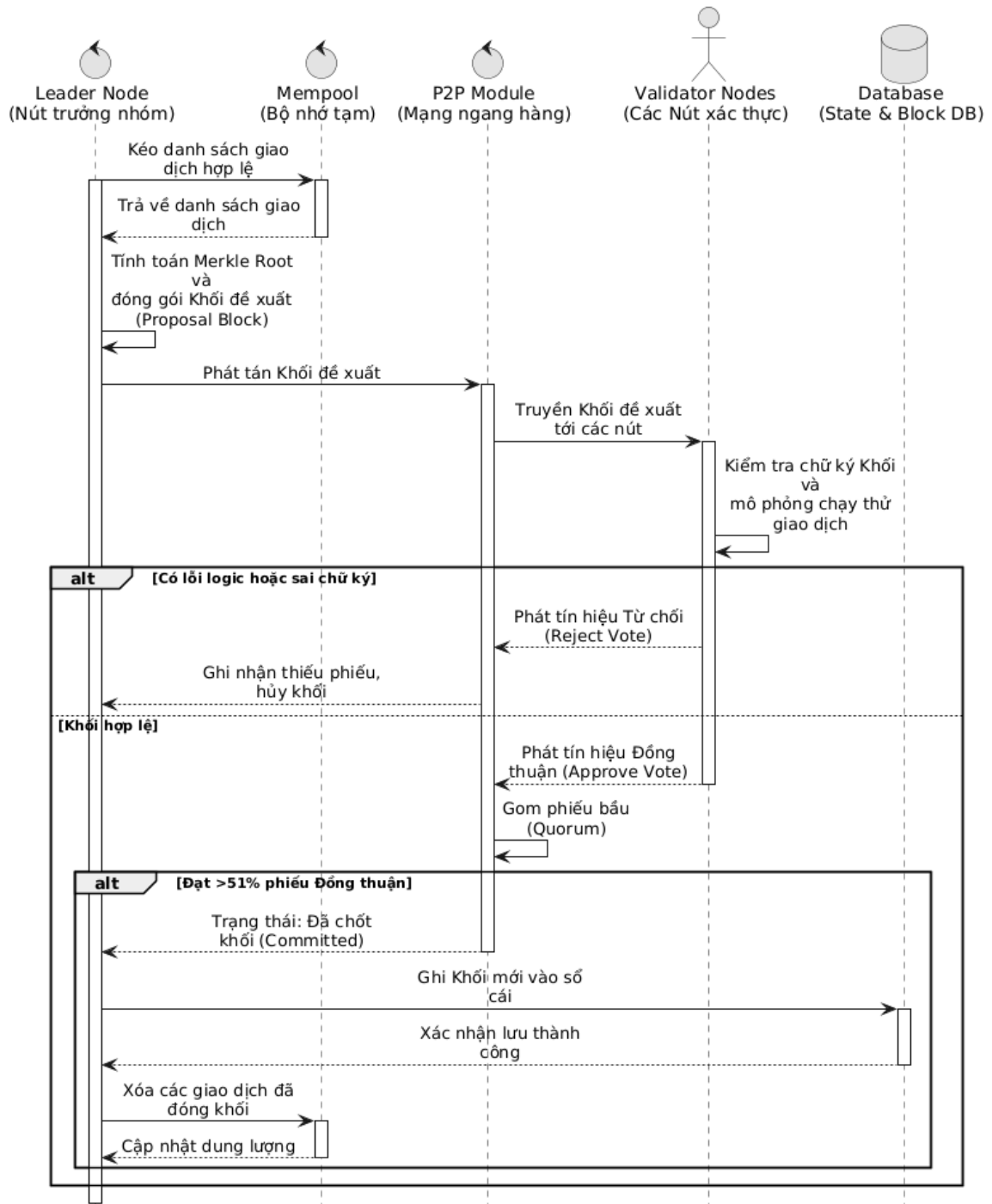
Hình 3.2: Sơ đồ Sequence khởi tạo mạng lưới Khoai Chain

## b) Tạo Node mới



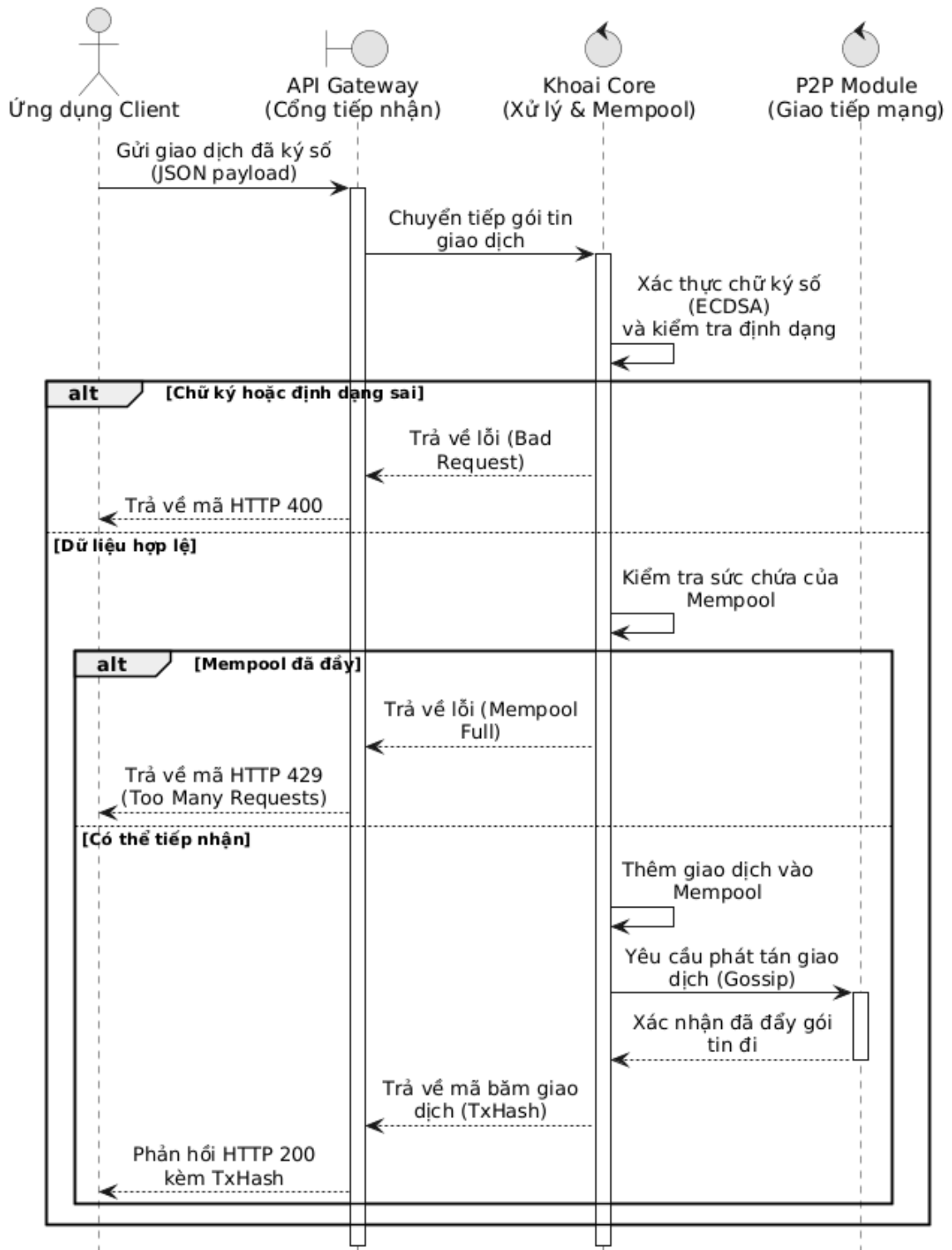
Hình 3.3: Sơ đồ Sequence tạo Node mới Khoai Chain

### c) Tạo Block



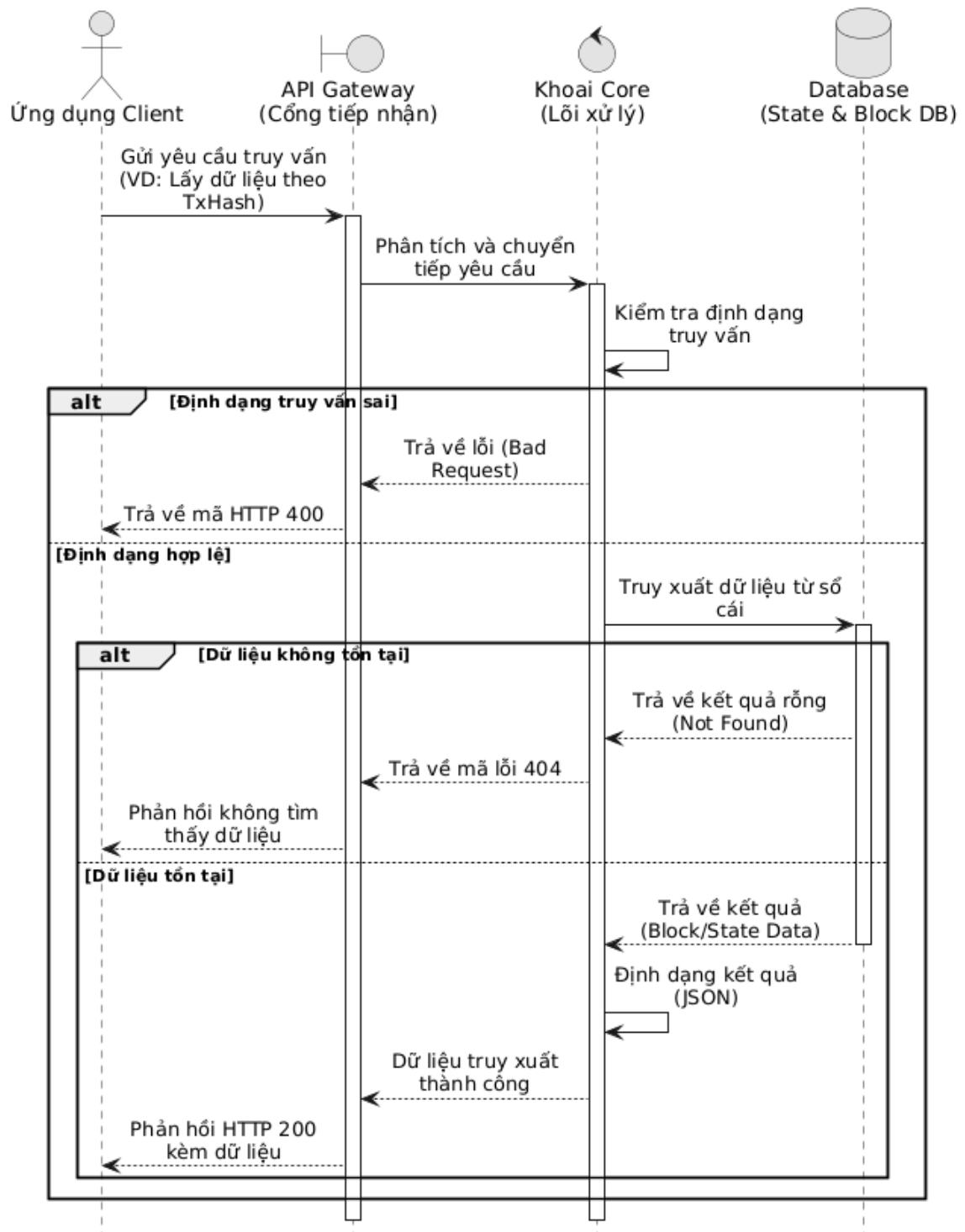
Hình 3.4: Sơ đồ Sequence tạo Block Khoai Chain

d) Gửi giao dịch



Hình 3.5: Sơ đồ Sequence gửi giao dịch Khoai Chain

e) Truy vấn sổ cái



Hình 3.6: Sơ đồ Sequence truy vấn sổ cái Khoai Chain

### 3.4. Yêu cầu phi chức năng

Các yêu cầu phi chức năng đặt ra các tiêu chuẩn về chất lượng, hiệu năng, tính bảo mật và khả năng vận hành của hệ thống Khoai Chain, đảm bảo hệ thống đáp ứng tốt các mục tiêu nghiên cứu và thực tiễn đặt ra.

- **Hiệu năng và Tốc độ xử lý (Performance):** Do hệ thống được thiết kế tinh gọn, loại bỏ các cơ chế đồng thuận nặng nề (như Proof of Work) và sử dụng cơ chế lưu trữ Key-Value nhúng trực tiếp, thời gian đóng gói và xác thực một khối mới (Block) phải đạt tốc độ cao (độ trễ dưới 1 giây khi chạy trên hạ tầng cục bộ). Tốc độ xử lý giao dịch (TPS - Transactions Per Second) phải đáp ứng mượt mà các bài toán nghiệp vụ nội bộ của doanh nghiệp vừa và nhỏ.
- **Tính bảo mật và Toàn vẹn dữ liệu (Security & Integrity):** Toàn bộ các giao dịch phát sinh từ ứng dụng client bắt buộc phải được ký số thông qua thuật toán chữ ký số đường cong Elliptic (ECDSA/Ed25519). Các nút trong mạng lưới phải thực hiện xác thực chữ ký và kiểm tra tính hợp lệ của gói tin trước khi cho phép đưa vào Mempool hoặc ghi vào sổ cái. Dữ liệu trên chuỗi phải đảm bảo tính bất biến, không thể giả mạo hay chỉnh sửa nhờ cơ chế liên kết mã băm (Cryptographic Hash Linking).
- **Khả năng mở rộng và Tính sẵn sàng của mạng P2P (Scalability & Availability):** Mạng lưới ngang hàng phải hỗ trợ cơ chế tự động khám phá và kết nối giữa các nút thông qua giao thức Gossip Protocol. Khi có một nút mới gia nhập hoặc ngắt kết nối đột ngột, các nút còn lại phải tự động cập nhật topology mạng và đồng bộ trạng thái sổ cái một cách nhanh chóng mà không làm gián đoạn toàn bộ hệ thống.
- **Tính tối ưu tài nguyên (Resource Efficiency):** Khác với các nền tảng blockchain công nghiệp đòi hỏi cấu hình phần cứng lớn, Khoai Chain phải tối ưu hóa mức tiêu thụ tài nguyên (CPU, RAM và dung lượng ổ cứng). Một node hệ thống phải có khả năng khởi chạy ổn định ngay cả trên các thiết bị máy tính cá nhân hoặc máy chủ ảo (VPS) cấu hình thấp, phục vụ hiệu quả cho môi trường thử nghiệm và giáo dục.
- **Tính độc lập và Dễ triển khai (Deployability & Maintainability):** Kiến trúc mã nguồn phải được tách biệt rõ ràng giữa tầng lõi (Core Blockchain) và bộ công cụ phát triển (SDK). Quá trình triển khai mạng lưới phải được tự động hóa thông qua công nghệ container hóa (Docker), cho phép người dùng khởi chạy và cấu hình toàn bộ hệ thống chỉ thông qua một vài thao tác lệnh cơ bản.

## Chương 4: Thiết kế Kiến trúc Khoai Chain

### 4.1. Kiến trúc tổng thể hệ thống

Hệ thống Khoai Chain được thiết kế theo mô hình kiến trúc tích hợp (Monolithic Node Architecture) nhằm tối ưu hóa tài nguyên phần cứng và giảm thiểu độ trễ mạng trong môi trường doanh nghiệp. Mỗi một Node trong mạng lưới là một thực thể độc lập chứa đầy đủ ba phân hệ cốt lõi: Module Mạng ngang hàng (P2P), Lõi Blockchain (Core), và Hệ thống Quản lý Hợp đồng thông minh (Smart Contract Manager).

Khác với kiến trúc phân rã của Hyperledger Fabric, Khoai Chain đóng gói toàn bộ quy trình từ tiếp nhận giao dịch, đóng khối, cho đến cập nhật trạng thái cơ sở dữ liệu vào một tiến trình duy nhất. Hệ thống sử dụng cơ sở dữ liệu dạng Key-Value (như LevelDB/BadgerDB) được nhúng trực tiếp, giúp loại bỏ hoàn toàn chi phí I/O khi phải giao tiếp với các container cơ sở dữ liệu bên ngoài. Toàn bộ kiến trúc được module hóa rõ ràng, cho phép dễ dàng mở rộng và bảo trì.

### 4.2. Thiết kế Module P2P (Peer, Server, Handler, Message)

Module P2P chịu trách nhiệm duy trì kết nối mạng lưới và đồng bộ hóa trạng thái toàn cục giữa các nút. Kiến trúc P2P được chia thành 4 thành phần chính:

- **Server & Peer:** Thành phần **Server** quản lý vòng đời của các kết nối TCP/IP hoặc WebSockets, liên tục lắng nghe các yêu cầu từ mạng. Mỗi kết nối với một nút khác được trừu tượng hóa thành một đối tượng **Peer**, lưu trữ định danh (Public Key) và trạng thái đồng bộ hiện tại.
- **Message:** Dữ liệu truyền tải giữa các nút được chuẩn hóa dưới định dạng JSON. Bản tin cơ sở bao gồm trường **Type** (như **MsgNewBlock**, **MsgSendChain**) và phần **Payload**. Thiết kế này giúp hệ thống dễ dàng phân loại và định tuyến gói tin.
- **Handler:** Đóng vai trò là bộ định tuyến logic. Khi nhận được bản tin, Handler sẽ kiểm tra tính hợp lệ của dữ liệu (Unmarshal) và kích hoạt các quy trình tương ứng. Đặc biệt, trong quá trình đồng bộ chuỗi (**MsgSendChain**), Handler áp dụng cơ chế xác thực lô (Batch Validation). Nó sẽ rà soát tính liên tục của mảng các khối nhận được (đảm bảo **PrevBlockHash** khớp hoàn toàn) trước khi cho phép ghi đè xuống cơ sở dữ liệu, ngăn chặn tuyệt đối tình trạng rác hóa cơ sở dữ liệu.

## 4.3. Thiết kế Core Blockchain

### 4.3.1. Cấu trúc dữ liệu Block và Transaction

Cấu trúc dữ liệu của Khoai Chain được thiết kế tối giản nhưng vẫn tuân thủ nghiêm ngặt các tiêu chuẩn mật mã học:

- **Transaction:** Là đơn vị thay đổi trạng thái cơ bản. Mỗi giao dịch chứa: ID (Mã băm giao dịch), **Timestamp**, và **Payload**. Trong Payload chứa thông tin định danh người gửi (**Sender**), tên Hợp đồng (**Contract**), Hàm gọi (**Function**), Tham số (**Args**), giá trị **Nonce** chống tấn công phát lại (Replay Attack), và Chữ ký số (**Signature**) sử dụng thuật toán Ed25519.
- **Block:** Đóng gói các giao dịch hợp lệ. Khối bao gồm **Height** (Độ cao chuỗi), **Timestamp**, danh sách các Transaction, và **PrevBlockHash** (Mã băm của khối liền trước). Mã băm của khối hiện tại (**Hash**) được tính toán bằng thuật toán SHA-256 dựa trên việc tuần tự hóa (Serialize) toàn bộ dữ liệu khối, sau đó lưu trữ dưới dạng chuỗi hệ thập lục phân (Hex string). Điểm đặc biệt của kiến trúc là khối gốc (Genesis Block) được cấu hình mang tính tất định tuyệt đối (Deterministic), đảm bảo mọi nút khởi chạy đều chung một điểm gốc toán học.

### 4.3.2. Luồng hoạt động của Mempool

Mempool (Memory Pool) là vùng nhớ đệm tạm thời chứa các giao dịch chưa được xác nhận. Luồng xử lý của Mempool được thiết kế cực kỳ chặt chẽ:

1. **Tiếp nhận & Xác thực sơ bộ:** Khi một giao dịch mới được gửi đến, Mempool kiểm tra định dạng chữ ký số Ed25519 bằng Public Key của người gửi. Nếu chữ ký sai lệch, giao dịch bị loại bỏ ngay tại viên mạng.
2. **Trạng thái chờ:** Các giao dịch hợp lệ được đẩy vào hàng đợi trong RAM để chuẩn bị đóng gói.
3. **Dọn dẹp tự động (Pruning):** Ngay khi một khối mới được đào thành công hoặc được đồng bộ từ mạng ngang hàng (**AddBlocks**), hệ thống sẽ quét qua danh sách các giao dịch nằm trong khối đó và gọi hàm **RemoveIncluded** để xóa chúng khỏi Mempool, giải phóng bộ nhớ và tránh việc thực thi trùng lặp.

## 4.4. Thiết kế hệ thống Smart Contract

Hệ thống Hợp đồng thông minh của Khoai Chain được thiết kế để tách biệt hoàn toàn trạng thái tính toán và trạng thái lưu trữ vĩnh viễn:

- **Contract Manager:** Là trái tim của hệ thống thực thi, quản lý trạng thái máy tự động. Manager cung cấp hai cơ chế cốt lõi: **ExecuteOnly** và **Commit**.
- **Luồng thực thi an toàn:** Khi xác thực một lô giao dịch từ mạng gửi về, hệ thống sử dụng **ExecuteOnly** để tính toán các logic nghiệp vụ. Kết quả thay đổi (State changes) chỉ được lưu tạm vào vùng nhớ đệm (**stagingState**). Nhờ vậy, nếu có bất kỳ giao dịch nào trong lô gây ra lỗi (như vi phạm quyền truy cập, sai logic hợp đồng), toàn bộ tiến trình có thể bị hủy bỏ mà không làm "bẩn" cơ sở dữ liệu chính.

- **Chốt trạng thái (Commit):** Chỉ khi toàn bộ các bước kiểm tra mật mã học và logic hợp đồng hoàn tất không có lỗi, hàm `Commit` mới được gọi. Lúc này, `stagingState` mới chính thức được ghi đè xuống cơ sở dữ liệu, đảm bảo tính nguyên tử (Atomicity) của hệ thống.
- **Router:** Đóng vai trò ánh xạ (mapping) tên hợp đồng và tên hàm từ giao dịch tới các đoạn mã Go đã được biên dịch tương ứng trong hệ thống.

## 4.5. Cơ chế đồng thuận

Do đặc thù hướng tới chuỗi doanh nghiệp nội bộ với các định danh có độ tin cậy nhất định, Khoai Chain không sử dụng mô hình bằng chứng công việc (PoW) nhằm tiết kiệm năng lượng. Thay vào đó, hệ thống tập trung vào tính toàn vẹn của chuỗi và luật chuỗi dài nhất (Longest Chain Rule) kết hợp với ràng buộc phủ hệ tuyến tính:

- **Đồng thuận dữ liệu gốc:** Các nút phải đạt được sự thống nhất tuyệt đối về khối Genesis. Bất kỳ sự khác biệt nào ở điểm khởi đầu đều khiến chuỗi bị mạng lưới từ chối.
- **Luật gắn kết khối (Linear Ancestry Enforcement):** Cơ chế đồng thuận cốt lõi nằm ở bước kiểm duyệt khối. Khi một Nút đề xuất một lô khối mới, các Nút xác thực không chỉ kiểm tra độ cao (`Height`), mà bắt buộc phải dò ngược mã băm (`PrevBlockHash`) để đảm bảo khối mới thực sự kế thừa từ chính khối đỉnh hiện tại (`LastHash`) lưu trong cơ sở dữ liệu.
- **Chống phân nhánh đa đầu (Multi-head Fork Prevention):** Nếu phát hiện khối mới không gắn với tổ tiên cục bộ, hoặc lô khối chứa mất xích đứt gãy, hệ thống sẽ tự động vô hiệu hóa toàn bộ quá trình đồng bộ lô (`Atomic Batch Rejection`). Điều này giải quyết triệt để nguy cơ hệ thống bị rẽ nhánh song song, đảm bảo Sổ cái luôn duy trì ở trạng thái là một chuỗi đơn nhất và minh bạch.

## Chương 5: Hướng dẫn sử dụng và Triển khai

### 5.1. Khởi tạo môi trường và cấu hình

Để hệ thống Khoai Chain vận hành trơn tru, người dùng cần chuẩn bị một môi trường tối giản bao gồm Docker Engine và Go runtime [9]. Toàn bộ thiết lập mạng lưới được tập trung vào một tệp cấu hình duy nhất nhằm giảm tải công việc của quản trị viên (DevOps).

- **Cấu hình mạng lưới qua `khoai-config.yaml`:** Tệp cấu hình này định nghĩa toàn bộ topology của mạng, bao gồm thông tin định danh của các nút (Node ID), địa chỉ IP, cổng giao tiếp P2P, và các tham số cốt lõi của chuỗi (như thời gian đóng khối tối đa, kích thước khối). Việc sử dụng định dạng YAML giúp các cấu hình trở nên trực quan và dễ dàng chỉnh sửa cho người dùng mới.
- **Quản lý phiên bản (Versioning):** Để tránh rủi ro phân nhánh mạng do lệch logic xử lý khối, hệ thống kiểm soát phiên bản lỗi chặt chẽ. Mỗi gói tin gửi đi qua mạng ngang hàng đều đính kèm cờ phiên bản (Version flag). Nếu phát hiện sự không tương thích (ví dụ: Node chạy Core v1.0 cố gắng kết nối với mạng v2.0), kết nối P2P sẽ bị từ chối ngay từ vòng bắt tay (handshake).

### 5.2. Chạy thử nghiệm bằng CLI và Docker

Một trong những điểm mạnh cốt lõi của Khoai Chain là trải nghiệm dành cho nhà phát triển (Developer Experience - DX) thông qua sự kết hợp giữa kiến trúc container và công cụ dòng lệnh (CLI).

- **Khởi chạy hạ tầng qua Docker:** Thay vì cài đặt từng thành phần cơ sở dữ liệu và biến môi trường phức tạp, người dùng chỉ cần một lệnh `docker-compose up -d`. Lệnh này sẽ tự động khởi tạo các container chứa lõi blockchain, ánh xạ (mount) các vùng nhớ dữ liệu cục bộ và thiết lập mạng nội bộ ảo (Docker network) cho các nút giao tiếp với nhau.
- **Tương tác qua KhoaiChain SDK (CLI):** Để tương tác với mạng lưới, người dùng không cần phải chui trực tiếp vào (exec) từng container. Bộ SDK được thiết kế dưới dạng công cụ CLI tiện lợi, cho phép thực thi các lệnh gọi nhanh (như truy vấn số dư, gửi giao dịch). Bản chất các lệnh CLI này hoạt động như một cầu nối client gọn nhẹ, kết nối và chạy lệnh trực tiếp trên hạ tầng Docker đang vận hành ngầm, mang lại cảm giác mượt mà và trực quan.

### 5.3. Kịch bản chạy thử nghiệm mẫu cho sinh viên

Nhằm phục vụ mục đích giáo dục, Khoai Chain cung cấp một kịch bản "Hello World" hoàn chỉnh, giúp sinh viên làm quen với kiến trúc blockchain chỉ trong vòng 15 phút:

1. **Bước 1 - Khởi động mạng:** Sinh viên sử dụng tệp `docker-compose.yaml` mẫu để dựng một mạng private nhỏ gồm 3 nút ngang hàng.

2. **Bước 2 - Khởi tạo Hợp đồng thông minh:** Sử dụng mã nguồn Go mẫu do hệ thống cung cấp (ví dụ: Hợp đồng quản lý chứng chỉ học tập), sinh viên sẽ đọc hiểu cấu trúc interface của hợp đồng.
3. **Bước 3 - Triển khai và Gửi giao dịch:** Sử dụng công cụ CLI để gửi một giao dịch cấp phát chứng chỉ lên mạng. Màn hình console sẽ hiển thị luồng log trực tiếp: Giao dịch được vào Mempool → Đóng khối → Đồng bộ P2P.
4. **Bước 4 - Truy vấn kết quả:** Sinh viên sử dụng CLI kết nối ngẫu nhiên vào một nút khác trong mạng để truy vấn thông tin chứng chỉ vừa tạo, qua đó trực tiếp kiểm chứng đặc tính đồng bộ và phân tán của sổ cái.

## 5.4. Các chức năng đang phát triển và Hạn chế hiện tại

Dù đã đáp ứng được mô hình blockchain riêng biệt tinh gọn, Khoai Chain ở phiên bản hiện tại vẫn tồn tại một số giới hạn kỹ thuật cần được khắc phục trong các giai đoạn nghiên cứu tiếp theo:

- **Khả năng tự động khám phá nút (Dynamic Node Discovery):** Mạng lưới hiện tại đang dựa vào cơ chế định tuyến tĩnh thông qua IP được cấu hình sẵn trong Docker. Trong tương lai, hệ thống cần tích hợp giao thức DHT (Distributed Hash Table) để các nút có thể tự động tìm thấy nhau trên môi trường Internet mở.
- **Cơ chế Rollback và Reorg phức tạp:** Hệ thống hiện tại đã ngăn chặn được các nhánh rác khi đồng bộ, nhưng nếu mạng bị đứt gãy vật lý (Network Partition) và tạo ra hai chuỗi song song dài bằng nhau, Khoai Chain chưa có cơ chế hồi phục (Reorg) toàn diện để tự động gỡ bỏ các khối bị sai lệch và đưa toàn mạng về một trạng thái đồng thuận duy nhất.
- **Bảo mật tính toán trong Smart Contract:** Môi trường thực thi Hợp đồng thông minh hiện chưa được cô lập tài nguyên tuyệt đối (Sandboxing). Một hợp đồng chứa vòng lặp vô hạn (Infinite Loop) do lập trình viên sơ suất có thể làm cạn kiệt tài nguyên CPU của tiến trình lõi. Cơ chế giới hạn tính toán (tương tự như "Gas Limit" của Ethereum) đang được nghiên cứu và phát triển.

# Kết luận

## Tóm tắt các chức năng đã chốt

Đồ án đã hoàn thành mục tiêu nghiên cứu và xây dựng thành công Khoai Chain – một nền tảng private blockchain tinh gọn, giải quyết triệt để sự chồng chéo của các hệ thống công nghiệp như Hyperledger Fabric. Tính đến giai đoạn hiện tại, hệ thống đã hoàn thiện và chốt hạ các chức năng cốt lõi sau:

- **Lõi Blockchain & Cơ sở dữ liệu nhúng:** Tự xây dựng toàn bộ cấu trúc dữ liệu Khối (Block) và Giao dịch (Transaction) từ con số không bằng ngôn ngữ Go [9]. Tích hợp thành công cơ sở dữ liệu Key-Value nhúng để lưu trữ trạng thái sổ cái, đảm bảo tốc độ truy xuất I/O tối ưu mà không cần phụ thuộc vào hệ quản trị cơ sở dữ liệu bên ngoài.
- **Mạng ngang hàng đồng bộ:** Triển khai thành công giao thức truyền tải bản tin (Message Protocol) và xử lý đồng bộ chuỗi. Đã giải quyết triệt để rủi ro phân nhánh đa đầu (Multi-head fork) bằng cơ chế kiểm duyệt phủ hệ tuyến tính nghiêm ngặt và đồng nhất hóa khối khởi thủy.
- **Quản lý Mempool & Hợp đồng thông minh:** Hoàn thiện luồng thực thi giao dịch an toàn với cơ chế tách biệt vùng nhớ đệm (**stagingState**). Hệ thống chỉ chốt (Commit) trạng thái sổ cái khi toàn bộ lô khối hợp lệ, loại bỏ rủi ro dữ liệu rác.
- **Triển khai Container hóa & CLI SDK:** Đóng gói thành công tiến trình Node vào môi trường Docker và xây dựng bộ công cụ giao diện dòng lệnh (KhoaiChain SDK), cho phép khởi chạy, quản trị và tương tác với mạng lưới chỉ qua các thao tác lệnh đơn giản.

## Định hướng hoàn thiện mã nguồn

Dựa trên nền tảng kiến trúc đã thiết lập, để Khoai Chain thực sự sẵn sàng cho môi trường triển khai thực tế (Production) của doanh nghiệp, mã nguồn cần tiếp tục được nghiên cứu và nâng cấp ở các khía cạnh sau:

- **Hoàn thiện cơ chế Reorg:** Bổ sung thuật toán Rollback tự động. Trong trường hợp mạng lưới gặp sự cố chia cắt sinh ra các nhánh tạm thời, hệ thống phải có khả năng tự động so sánh, loại bỏ các khối thuộc nhánh ngắn hơn và đồng bộ lại theo nhánh chính thức khi mạng kết nối lại.
- **Bảo vệ môi trường thực thi Hợp đồng:** Cài đặt cơ chế giới hạn tài nguyên (như áp dụng `context.WithTimeout` trong Go) để tự động ngắt các hợp đồng thông minh bị lỗi vòng lặp vô hạn, ngăn chặn tình trạng cạn kiệt tài nguyên CPU (Denial of Service) của Node tiếp nhận.
- **Tối ưu hóa khả năng lưu trữ:** Khi hệ thống vận hành lâu dài, kích thước sổ cái sẽ phình to. Cần nghiên cứu phát triển cơ chế nén hoặc dọn dẹp dữ liệu lịch sử nhưng vẫn giữ lại các bằng chứng mật mã để đảm bảo các Node cấu hình thấp vẫn có thể tham gia lưu trữ mà không bị tràn bộ nhớ đệm.

- **Cơ chế bảo mật dữ liệu riêng tư:** Nâng cấp kiến trúc luồng dữ liệu để hỗ trợ các giao dịch ẩn danh giữa một nhóm Node cụ thể trong mạng. Điều này giúp các doanh nghiệp trong cùng chuỗi có thể giao dịch nội bộ mà không làm lộ chi tiết hợp đồng ra toàn bộ phần còn lại của mạng lưới.
- **Phát triển giao diện API kiểm thử nhanh:** Tích hợp một module máy chủ web hạng nhẹ mở cổng giao tiếp HTTP/RESTful (tương tự kiến trúc của FastAPI) trực tiếp vào Node. Phân hệ này sẽ cung cấp một giao diện trực quan giúp các nhà phát triển dễ dàng gửi giao dịch, gọi thử các hàm Smart Contract và xem log trạng thái thay đổi theo thời gian thực mà không cần phụ thuộc hoàn toàn vào giao diện dòng lệnh.

## Tài liệu tham khảo

- [1] A. M. Antonopoulos, *Mastering Bitcoin: Programming the Open Blockchain*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2017.
- [2] E. Androulaki et al., "Hyperledger fabric: a distributed operating system for permissioned blockchains," in *Proceedings of the Thirteenth EuroSys Conference*, Porto, Portugal, 2018, pp. 1-15.
- [3] L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382-401, 1982.
- [4] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proceedings of the Third Symposium on Operating Systems Design and Implementation*, New Orleans, LA, 1999, pp. 173-186.
- [5] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *2014 USENIX Annual Technical Conference*, Philadelphia, PA, 2014, pp. 305-319.
- [6] A. Demers et al., "Epidemic algorithms for replicated database maintenance," in *Proceedings of the sixth annual ACM Symposium on Principles of distributed computing*, Vancouver, BC, Canada, 1987, pp. 1-12.
- [7] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. London, UK: Pearson, 2017.
- [8] D. J. Bernstein et al., "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77-89, 2012.
- [9] A. A. A. Donovan and B. W. Kernighan, *The Go Programming Language*. Boston, MA: Addison-Wesley Professional, 2015.